N2SCFTDB

$\mathcal{N} = 2$ theories

Mathematical background and implementation reference

One reference, organized by package

Exact Lie-algebra data, anomaly checks, superconformal indices, Coulomb-branch indices and spectra, theory properties, database persistence, and the planned Higgs-branch extension.

Native implementation options

This update adds the native C++ full-index backend, shared-library LiE, backend selection, build commands and GUI engine logging in Section 7. The mathematical conventions and algorithms are unchanged. Earlier package descriptions, validation and benchmark snapshots retain their dates and scopes.

Snapshot	Base reference: 8 September 2026. Database and validation update: 9 September 2026. Index/cache update: 10 September 2026; `04bbe21`. Property/database: 12 September. GUI/concurrency: 14 September. Index sectors/cache-file arguments: 15 September. FORM and Settings deletion: 16 September. Search/export and production schema: 19 September 2026. Native implementation options: 19 October 2026.
Consolidates	`index/n2_theory_index_Mathematical_Background.pdf` and the previous `output/pdf/n2_implementation_reference_summary.pdf`.
Status	Anomaly, index and search tabs, CSV export and Settings deletion are implemented; basic properties and index calculation remain separate. Unrelated mathematics and benchmark chapters retain their dated snapshots. Higgs/Hall–Littlewood material is explicitly marked as proposed, not implemented.
Editing scope	Implementation reference only. The companion index-background PDF and its mathematical source are unchanged.

1 Project map and conventions

Package	Responsibility	Read here
`anomalies`	Cartan types, representations, full/half hypers, beta functions, local and conventional mod-two gauge anomalies.	§2
`index`	FORM expansion, LiE partial decompositions and SQLite singlet cache; Coulomb PE/PL and full-index projection; future Higgs/flavor refinement.	§3
`common`	Property orchestration, flavor blocks, central charges, exact-number and FORM helpers, JSON and MySQL persistence.	§4
`gui`	Editable settings and gauge lists, parallel SCFT imports, cache preparation, search/CSV export and persistent logs.	§5
`test`	Unit and backend regression tests, precision and serialization checks, optional live database integration.	§6

How data moves

Theory dictionary / JSON $\longrightarrow$ validated gauge and hyper data
$\longrightarrow$ anomaly and conformality checks
$\longrightarrow$ basic properties and optional MySQL insertion
$\longrightarrow$ separate index / Coulomb-spectrum worker
$\longrightarrow$ fill missing data or upgrade known lower cutoffs

Standalone index functions are also callable directly. The property and database workflows impose the SCFT-candidate checks; a gauge-invariant counting calculation alone is not a proof that an input defines an SCFT.

Notation used throughout

$G = \prod_{a=1}^{s} G_a$	Compact, simply connected, semisimple gauge group; no abelian gauge factor or global quotient is implemented.
$\mathcal{R}_h = \boxtimes_a R_{ha}$	One product-group hyper representation; n_h is its number of copies. Omitted factors are singlets.
$\epsilon_h = 1, \frac{1}{2}$	Weight of a full or half hyper, respectively. Beta functions use $2\epsilon_h T$, not $\epsilon_h T$.
$E, R, r; j_1, j_2$	Scaling dimension, $SU(2)_R$ weight, $U(1)_r$ charge, and Lorentz Cartan weights in the chosen right-index convention.
t, y, u	Project full-index fugacities; the common literature variable $v = u^2$.
x, τ, q	Coulomb grading, proposed HL grading, and an internal FORM integer-power variable. This q is not a conventional full-index fugacity.

> **What "spectrum" means here**
>
> The Coulomb spectrum is the multiset of dimensions of independent chiral-ring generators, with repetitions. Index coefficients count all graded ring elements, including products. Neither is the spectrum of massive BPS particles or the full operator spectrum of the SCFT.

2 Package `anomalies`

2.1 Root data and representation invariants

MODULE `anomalies/lie_algebra.py`

`SimpleLieAlgebra` collects Sage root, weight and character objects together with the rank, dimension, dual Coxeter number and adjoint Dynkin labels. The cached constructor is `get_lie_algebra`. The accepted finite types are A_r $(r \geq 1)$, B_r, C_r $(r \geq 2)$, D_r $(r \geq 4)$, $E_{6,7,8}$, F_4, and G_2. Use A_1 for B_1/C_1, A_3 for D_3, and $A_1 \times A_1$ for D_2.

For a dominant integral highest weight λ, Dynkin labels and dimensions are

$$\lambda = \sum_i a_i \omega_i, \qquad a_i = \langle \lambda, \alpha_i^\vee \rangle \in \mathbb{Z}_{\geq 0}, \tag{2.1}$$

$$\dim V_\lambda = \prod_{\alpha>0} \frac{(\lambda+\rho,\alpha)}{(\rho,\alpha)}, \qquad \rho = \frac{1}{2}\sum_{\alpha>0}\alpha. \tag{2.2}$$

The algebra dimension is $\dim \mathfrak{g} = \operatorname{rank}\mathfrak{g} + |\Phi|$, and the adjoint highest weight is the highest root. `representation_dimension` uses the degree of the Sage Weyl character, implementing (2.2). Source: Sage root-system and Weyl-character documentation [1, 2].

To compensate for the ambient root-length convention used by Sage, the code uses

$$C_2(\lambda) = \frac{(\lambda, \lambda+2\rho)}{\ell^2_{\text{long}}}, \qquad T(R) = \frac{\dim R}{\dim \mathfrak{g}} C_2(R). \tag{2.3}$$

If long roots have squared length two, the first denominator is two. Thus $C_2(\text{adj}) = T(\text{adj}) = h^\vee$ and $T(\mathbf{N}) = 1/2$ for $SU(N)$. The routines are `quadratic_casimir` and `dynkin_index`. The physics normalization is fixed by $\mathrm{Tr}_R(T^a T^b) = T(R)\delta^{ab}$.

Names, conjugation and reality

`named_representation_labels` supports common names such as fundamental, antifundamental, adjoint, singlet, classical spinors and selected two-index representations. Explicit Dynkin labels are the general interface. Numbering is Sage's finite-Cartan numbering; do not copy a label tuple from another reference without checking its node convention.

$$\lambda^* = -w_0\lambda, \qquad \nu(R) = \int_G \chi_R(g^2)\, d\mu(g) \in \{+1, 0, -1\}. \tag{2.4}$$

Here $\nu = +1, 0, -1$ means real, complex, pseudoreal. `conjugate_dynkin_labels` uses the dual character; `representation_reality` uses its Frobenius–Schur indicator. For an irreducible external tensor product, $\nu(\boxtimes_a R_a) = \prod_a \nu(R_a)$. Source: Sage Weyl characters [2]; representation and index conventions can be cross-checked with LieART 2.0 [4].

2.2 Theory data, hypermultiplets and the beta function

MODULE `anomalies/check_n2_anomalies.py`

The general input parser constructs `GaugeFactorData(factor_id,algebra)` objects and simple/product hyper data. Counts are exact nonnegative integers. Malformed types, labels or representations are rejected before calculation. The theory dictionary supports either one simple factor or a list of named gauge factors with a representation for each charged factor.

In $\mathcal{N}=1$ language, a full hyper in $\mathcal{R}$ contains chirals in $\mathcal{R}\oplus\overline{\mathcal{R}}$. A half hyper contains one chiral and is allowed only when the *entire* gauge representation is pseudoreal. Consequently $\mathbf{2}\boxtimes\mathbf{2}$ of $SU(2)^2$ is real and cannot be a single half hyper, whereas $\mathbf{2}\boxtimes\mathbf{2}\boxtimes\mathbf{2}$ is pseudoreal and can. This is validated rather than inferred from one factor alone. Source: Bhardwaj–Tachikawa, §2, Eqs. (2.1)–(2.4) [3].

For factor a, spectator dimensions multiply the effective number of charged degrees of freedom. In the normalization of (2.3),

$$b_{0,a} = 2h_a^\vee - \sum_h n_h\,(2\epsilon_h)\,T_a(R_{ha})\prod_{b\neq a}\dim R_{hb}. \tag{2.5}$$

The implementation uses the literal coefficient two for full hypers and one for half hypers. $b_{0,a}=0$ for every simple factor is its one-loop conformality condition; $b_{0,a}>0$ denotes asymptotic freedom in this convention. Source: Bhardwaj–Tachikawa, Eqs. (2.5)–(2.8) [3].

Checks that expose normalization mistakes

For $SU(N)$ with N_f full fundamental hypers,

$$b_0 = 2N - N_f. \tag{2.6}$$

For one full bifundamental $(\mathbf{N},\mathbf{M})$ of $SU(N)\times SU(M)$, its contributions to the two beta-function subtractions are M and N. For a half hyper in an $SU(2)$ fundamental, the subtraction is $1/2$. These follow directly from (2.5), not from a new convention for T.

> **Two different uses of "index"**
>
> The Dynkin index $T(R)$ is a quadratic trace invariant. It is unrelated to the superconformal index $\mathcal{I}$. LieART's commonly tabulated quadratic index $\ell(R)$ in the old project comparison is $2T(R)$; convert the convention before comparing tables. Root/node numbering must be converted independently.

Scope of the candidate flag

The anomaly checker combines valid matter, gauge-anomaly cancellation and vanishing beta functions into a `lagrangian_scft_candidate` result. It does not reconstruct a Seiberg–Witten geometry, identify S-dual theories, classify line operators, or handle non-Lagrangian matter sectors.

2.3 Local and conventional global gauge anomalies

For a left-handed Weyl fermion in R, the local cubic anomaly tensor is

$$d_R^{abc} = \mathrm{Tr}_R\Big(T^a\{T^b, T^c\}\Big), \qquad d_{\overline{R}}^{abc} = -d_R^{abc}. \tag{2.7}$$

Full hypers cancel between R and $\overline{R}$; self-conjugate real or pseudoreal representations have zero cubic tensor. The adjoint gauginos are in a real representation. Thus local anomaly cancellation follows structurally for the allowed matter schema; the checker need not tabulate every cubic invariant. For semisimple product groups, mixed tensors with one generator from another factor vanish by $\mathrm{Tr}\, T^a = 0$. Source: Bilal, discussion of chiral fermions, conjugate representations and anomaly traces [5]; matter restrictions in [3].

The mod-two test

For factors with the conventional four-dimensional Witten anomaly, the number of anomalous Weyl representations must be even. The code checks A_1, all supported C_r, and $B_2 \simeq C_2$. In its normalization the parity is

$$\mathfrak{a}_a = \sum_{h\ \text{half}} n_h\,[2T_a(R_{ha})] \prod_{b\neq a} \dim R_{hb} \pmod 2. \tag{2.8}$$

Full hypers contribute an even number automatically. The integer entering this test is $2T$, not the full-hyper beta contribution with an additional factor of two. A nonzero parity makes the gauge theory inconsistent within this setup. Source: Witten [6]; Wang–Wen–Witten, §2.3 [7]; symplectic-group discussion in [3, 8].

For $SU(2)$ isospin j, a useful independent formula is

$$T_j = \frac{1}{3}j(j+1)(2j+1), \qquad 2T_j = \frac{2}{3}j(j+1)(2j+1). \tag{2.9}$$

It is odd for $j = 2k + 1/2$, $k \in \mathbb{Z}_{\geq 0}$: $j = 1/2, 5/2, 9/2, \ldots$. One fundamental half hyper is anomalous, two are safe. A single $SU(2)^3$ trifundamental half hyper gives four doublets to each factor and is safe under this conventional test, although it is not by itself conformal.

> **Not an all-background anomaly classification**
>
> The project assumes the ordinary spin-theory setting and simply connected groups. It does not implement the newer non-spin $SU(2)$ anomaly associated with isospin $4k + 3/2$, global quotients, discrete theta angles or general bordism anomalies. Flavor 't Hooft anomalies need not vanish and are not gauge-anomaly rejection criteria. See [7, 8] for the distinction.

2.4 Independent $SU(N)$ formula checks

This section retains the useful representation-theory cross-checks from the older reference. They are derived checks, not a second implementation backend. For Dynkin labels $(a_1, \ldots, a_{N-1})$, define row lengths and box count by

$$\ell_i = \sum_{k=i}^{N-1} a_k \quad (i < N), \qquad \ell_N = 0, \qquad B = \sum_{i=1}^{N} \ell_i. \tag{2.10}$$

Specializing the Weyl dimension and highest-weight Casimir formulas gives

$$\dim R = \prod_{1 \le i < j \le N} \frac{\ell_i - \ell_j + j - i}{j - i}, \tag{2.11}$$

$$C_2(R) = \frac{1}{2}\left[NB + \sum_{i=1}^{N} \ell_i(\ell_i + 1 - 2i) - \frac{B^2}{N}\right], \tag{2.12}$$

$$T(R) = \frac{\dim R}{N^2 - 1} C_2(R). \tag{2.13}$$

Source: Direct specialization of (2.2) and (2.3); compare Sage characters [2] and representation tables in [4].

$SU(N)$ representation	Dimension	C_2	T
Fundamental	N	$(N^2-1)/(2N)$	$1/2$
Adjoint	N^2-1	N	N
Two-index symmetric	$N(N+1)/2$	$(N-1)(N+2)/N$	$(N+2)/2$
Two-index antisymmetric	$N(N-1)/2$	$(N+1)(N-2)/N$	$(N-2)/2$

One symmetric plus one antisymmetric full hyper therefore gives $2T_{\text{sym}} + 2T_{\text{asym}} = 2N$ and $b_0 = 0$. For $N = 2$ the antisymmetric is a singlet, with zero T.

Function-to-equation map

Routine	Mathematical object
`get_lie_algebra`, `validate_dynkin_labels`	Root data and (2.1)
`representation_dimension`	(2.2), checked by (2.11)
`quadratic_casimir`, `dynkin_index`	(2.3)
`conjugate_dynkin_labels`, `representation_reality`	(2.4)
Gauge-factor beta calculation	(2.5)
Gauge-anomaly checks	(2.7), (2.8)

3 Package `index`

Execution backend update

Full-index calculation now defaults to native C++ expansion and projection. Section 7 documents the current options. The retained FORM expansion and executable-LiE descriptions below explain the reference path; Coulomb PE/PL continues to use FORM/TFORM.

3.1 Full-index convention and single-letter functions

MODULE `index/n2_theory_index.py`

In radial quantization the chosen right-handed supercharge restricts the trace to

$$\delta_R = E - 2j_2 - 2R + r = 0, \tag{3.1}$$

$$\mathcal{I}(t,y,u) = \mathrm{Tr}_{\delta_R=0}(-1)^F t^{2(E+j_2)} y^{2j_1} u^{-2(r+R)}. \tag{3.2}$$

This is the literature convention with $v = u^2$. It is a signed protected trace, not a positive count of every short multiplet: cancellations and recombination mean the full operator spectrum cannot generally be recovered uniquely. Source: Gadde–Pomoni–Rastelli, Eqs. (5.4)–(5.5) [9].

Writing $D(t,y) = (1-t^3y)(1-t^3/y)$, the scalar coefficients of the vector and half-hyper characters are

$$f_V(t,y,u) = \frac{t^2u^2 - t^4u^{-2} - t^3(y+y^{-1}) + 2t^6}{D(t,y)}, \tag{3.3}$$

$$f_H(t,y,u) = \frac{t^2/u - t^4u}{D(t,y)}. \tag{3.4}$$

The denominator counts the two allowed spacetime derivatives; fermionic letters and equations of motion produce the numerator signs. Source: Gadde–Pomoni–Rastelli, Eqs. (5.21)–(5.22), with $v = u^2$ [9].

For gauge variables $g = (g_1, \ldots, g_s)$, the complete letter function is

$$L = f_V \sum_a \chi_{\mathrm{adj}_a}(g_a) + f_H \left[\sum_{h \text{ full}} n_h (\chi_{\mathcal{R}_h} + \chi_{\overline{\mathcal{R}_h}}) + \sum_{h \text{ half}} n_h \chi_{\mathcal{R}_h} \right]. \tag{3.5}$$

Here $\chi_{\mathcal{R}_h}(g) = \prod_a \chi_{R_{ha}}(g_a)$. The conjugate of a product representation conjugates every factor simultaneously: $(R,S) \mapsto (\overline{R},\overline{S})$. A full bifundamental is not four independent choices of conjugation. For a self-conjugate full hyper, the two equal characters still give multiplicity two.

Flavor-unrefined does not mean flavor-singlet

All flavor fugacities are currently set to one. Flavor representations are replaced by their dimensions, while gauge representations are kept until singlet projection. Charged flavor operators are counted; the code does not project them out. Spin and R-symmetry fugacities y, u are still present.

3.2 Plethystic expansion and FORM truncation

For a letter function with no degree-zero term, define

$$\mathrm{PE}[L] = \exp\left[\sum_{k\geq 1}\frac{1}{k}L(t^k, y^k, u^k; g_1^k, \ldots, g_s^k)\right], \qquad \mathcal{I} = \int_G d\mu(g)\ \mathrm{PE}[L]. \tag{3.6}$$

Adams substitution acts on every fugacity and character, but leaves numerical multiplicities and the signed letter coefficients as coefficients. The Haar measure is normalized so $\int_G 1 = 1$. Source: Plethystics [10]; gauge-index construction in [9].

`_character_basis` assigns formal character functions $C_p(k) = \chi_{R_p}(g_a^k)$ to simple-factor irreps. `_build_form_program` constructs the truncated exponent

$$A_D = \left[\sum_{k=1}^{\lfloor D/2\rfloor}\frac{L(t^k, y^k, u^k; g^k)}{k}\right]_{t\leq D}. \tag{3.7}$$

Every letter begins at t^2, so $k > \lfloor D/2\rfloor$ and particle number $m > \lfloor D/2\rfloor$ cannot contribute. Derivatives are expanded as

$$D(t,y)^{-1} = \sum_{p,q\geq 0} t^{3(p+q)}y^{p-q}; \qquad p,q \leq \lfloor D/3\rfloor \tag{3.8}$$

is a sufficient rectangular bound before final degree truncation. FORM declares t with maximum power D, so excess combined powers are discarded throughout the calculation. Negative powers of u and y are allowed.

The auxiliary-variable recurrence

The baseline recurrence starts with zA_D and successively substitutes

$$z \longmapsto 1 + \frac{zA_D}{m}, \qquad m = 2, \ldots, M, \quad M = \lfloor D/2\rfloor. \tag{3.9}$$

After step m, the expression is $\sum_{j=1}^{m-1} A_D^j/j! + zA_D^m/m!$. Setting $z = 1$ and adding the vacuum term gives $\sum_{j=0}^{M} A_D^j/j!$ exactly through degree D. This is the algebraic reason for the generated FORM substitutions, rather than an approximation requiring a convergence assumption. Source: Recurrence derived from the source; FORM syntax and truncation conventions in the supplied FORM 5 manual [12], with software background in [13].

`run_form` is imported from `common/form_utils.py`. `FormExpansionCache.parse_form_output` in `index/form_expansion_cache.py` converts rational coefficients, fugacity exponents and character powers into `IndexFormTerm` records. `get_expansion` reuses their SQLite serialization when the complete raw FORM program is already cached. The numerical coefficients remain exact rationals, including cancellations generated by the index.

Degree-bounded FORM exponential: current implementation

The 16 September implementation in `_build_form_program` evaluates the same finite exponential as the auxiliary-variable recurrence above, but avoids products that cannot survive the requested cutoff. Let N denote that cutoff (called D in the earlier formulas), and write

$$A = \texttt{itotal} = \sum_{d=2}^{N} A_d t^d, \qquad [P]_N = \sum_{d \le N} [t^d] P \, t^d, \qquad M = \lfloor N/2 \rfloor.$$

The coefficients A_d contain Laurent monomials in y, u and formal gauge characters. They contain no t. All powers of t are nonnegative and the exponent starts at degree two. The target before gauge projection is $[\exp(A)]_N$. The derivation below follows the project's generated program.

Step 1: construct the derivative series and plethystic exponent. The expression `J` is the truncated product of geometric series for $(1-t^3y)^{-1}(1-t^3/y)^{-1}$. Substituting the vector and hyper letters into `itotal` gives

$$A = \left[\sum_{j=1}^{M} \frac{1}{j} \left(f_V(t^j, y^j, u^j) \sum_{p \in V} C_p(j) + f_H(t^j, y^j, u^j) \sum_{m} N_m \prod_{p \in m} C_p(j) \right) \right]_N .$$

Here j is the Adams index. It differs from the outer Taylor-power index i below. Integer multiplicities N_m are unchanged by Adams substitution. The bound on j follows from $2j \le N$; the derivative rectangle uses `order//3`. The declaration `t(:N)` remains a final degree guard.

Step 2: organize the exponent and initialize the active power.

```
Bracket t;
.sort
Skip J,itotal;
L I=z*itotal;
.sort
```

`Brackett` groups equal t powers. A reference `itotal[t^d]` returns A_d, so multiplication must restore `t^d` explicitly. `SkipJ,itotal;` preserves those expressions, including the bracket information, for the current module. They remain readable on the right-hand side. Repeat Skip after each module boundary; it is not a permanent freeze [26, entries "bracket", "skip", and Chapter 9].

Initially $I = zA$. The symbol z marks the one Taylor order still eligible to generate the next order; unmarked terms have already been accumulated. Neither z nor the temporary symbol w is a physical fugacity.

One exponential step, line by line

For example, at cutoff $N = 8$ the following is the actual emitted loop and finalization block; Python substitutes the integer bounds before running FORM:

```
#do i=2,4
  Skip J,itotal;
  #do k=2,6
    if (count(t,1) == 'k');
      id z=1+w*sum_(idx1,2,8-'k',itotal[t^idx1]*t^idx1)/'i';
    endif;
  #enddo
  id z=1;
  id w=z;
  .sort:step 'i';
#enddo

L result=1+I;
id z=1;
.sort
Print result;
.end
```

Step 3: select the current term's degree. The inner preprocessor loop creates one branch for each $k = 2, \ldots, N-2$. `count(t,1)` tests the current term's t degree. An unmarked term is unchanged because it has no z to replace. On a marked term zct^k, the substitution has the exact effect

$$zct^k \longmapsto ct^k + \frac{wc}{i} \sum_{d=2}^{N-k} A_d t^{k+d}.$$

The first branch retains the old Taylor contribution. The second produces the next contribution, dividing by i to turn $(i-1)!$ into $i!$. Its summation bound is the condition $k + d \leq N$, enforced before forming the product rather than only when an excessive power is normalized away.

Step 4: protect newly generated terms. New terms carry w, not z. A later k branch in the same outer iteration can encounter one of their new degrees, but its `idz=...` cannot expand them again. Reusing z immediately would allow a single Taylor step to multiply a newly produced term more than once.

Step 5: finalize old terms, then mark the new ones. `idz=1;` removes any surviving old marker, including terms at degrees $N-1$ and N. Then `idw=z;` makes only the new generation active. Reversing this order would erase the new marker. Finally `.sort:step'i';` collects equal terms and ends the module; the step label is diagnostic. The next iteration reinstates Skip for its own module.

Why the bounds and factorials are exact

For $2 \le i \le M$, induction on the previous step gives

$$\begin{aligned} I_1 &= zA, \\ I_2 &= A + z[A^2/2!]_N, \\ I_3 &= A + [A^2/2!]_N + z[A^3/3!]_N, \\ I_i &= \sum_{r=1}^{i-1} [A^r/r!]_N + z[A^i/i!]_N. \end{aligned}$$

Only the last summand has a marker. Step $i+1$ retains it without a marker and multiplies it by $A/(i+1)$, keeping precisely the products of total degree at most N. This proves the invariant with exact rational coefficients.

Why the inner loop stops at $N-2$. Every term of A has degree at least two, so an old term of degree k can produce a retained new term only when $k+2 \le N$. Terms of degree $N-1$ and N are already present and are retained by `idz=1;`. Thus $N-2$ is a bound on terms that can be multiplied again, not a reduced output cutoff.

Why the outer loop stops at $\lfloor N/2 \rfloor$. A^i has minimum degree $2i$. No higher power can contribute. Adding the vacuum and removing the last marker therefore gives

$$\texttt{result} = 1 + \sum_{r=1}^{M} [A^r/r!]_N = [\exp(A)]_N.$$

The public API returns 1 immediately for orders zero and one. Orders two and three need only $1 + A$; Python omits the multiplication loop. These bounds use the full-index grading; a Hall–Littlewood letter beginning at degree one would require different bounds.

Worked example, $N = 6$. Set $A = at^2 + bt^3$, with commuting exact coefficients that may themselves be formal characters. Then

$$\begin{aligned} I_2 &= at^2 + bt^3 + z\left(\frac{a^2}{2}t^4 + abt^5 + \frac{b^2}{2}t^6\right), \\ I_3 &= at^2 + bt^3 + \frac{a^2}{2}t^4 + abt^5 + \frac{b^2}{2}t^6 + z\frac{a^3}{6}t^6. \end{aligned}$$

In step three, only the marked degree-four term can multiply again. The degree-five and degree-six contributions become unmarked without being lost. The final coefficient of t^6 is $b^2/2 + a^3/6$, as in the ordinary exponential.

Validation and cache effect. `test/test_form_degree_bound.py` checks 42 low-order formal expansions (seven cases at six cutoffs) and one connected bifundamental at order 18 against the former multiplication. The 16 September recorded full run passed 357 tests in 64.117 s, with no skips. A matched, sequential FORM-only comparison at order 18 took 2.236 s before and 0.585 s afterward (3.82 times faster), with exactly the same 28,154 formal terms. This is one measurement, not a universal speedup. Evidence is retained in `output/benchmarks/degree_bounded_form_20260916.json`. The changed raw program has a new FORM-cache key. Existing rows remain valid for their original programs, and character-cache entries remain reusable.

FORM or TFORM: execution controls and CPU allocation

The symbolic program and exact projection algorithm are unchanged by choosing TFORM. `run_form` in `common/form_utils.py` validates `form_threads` as an exact positive integer. With one thread it invokes ordinary FORM; above one it invokes the separate TFORM executable:

```
form -q program.frm
tform -w32 -q program.frm
```

Both run in a temporary working directory. Missing executables, timeout, nonzero exit status or nonempty stderr are reported as errors. The production wrapper captures complete stdout in Python; selecting TFORM does not make parsing, projection or database writes threaded.

Full-index and Coulomb-index/PE/PL entry points propagate `form_threads=1` and `tform_executable ="tform"`. The full-index and database-index CLIs expose `--form-threads` and `--tform-executable`, alongside the existing FORM path and timeout. For example, from the project root:

```
sage -python -m index.n2_theory_index theory.json \
  --order 18 --form-threads 32 --tform-executable tform \
  --processes 1 --char-cache-database /tmp/characters.db \
  --form-cache-database /tmp/expansions.db
```

Use a new empty FORM cache when measuring an expansion. The raw program is the cache key and does not include the engine, its version or thread count. A hit can therefore reuse a prior engine's parsed result without running TFORM. Stored-sector reuse can bypass the expansion too.

Two levels of parallelism. The index coordinator uses the saved CPU budget P and FORM worker count T together. For $C > 0$ selected theories:

$$W_{\text{index}} = \min(C, \max(1, \lfloor P/T \rfloor)) .$$

Here CPU cores `-1` resolves to the logical CPU count, or one if unavailable. On a 32-CPU host, $T = 1, 8, 32$ gives at most $32, 4, 1$ simultaneous theory workers. If $T > P$, one theory still runs with the requested T threads; the setting does not silently clamp TFORM. Each theory worker keeps inner LiE/character work at one process. Anomaly enumeration and CSV export use their own CPU allocation rules. These are implementation contracts, not mathematical identities.

Memory and scope. A single expansion with many TFORM workers can still consume several GiB. Requested workers and observed OS thread counts differ; the recorded 32-worker runs reached up to 63 OS threads. The measured RSS is the complete TFORM process, excluding the Sage parent, page cache and later parsing/projection. The controls allow concentrating CPU work on one index; they do not establish a RAM bound.

Recorded checks, 16 September. Connected/disconnected full indices, fractional Coulomb PE/PL, engine-independent cache reuse, settings reload and worker allocation were checked in the implementation run: 340 passed, 38 skipped. See `test/test_form_threads.py`. The measurements on the next pages are separate one-run-per-theory experiments, not those regression tests.

TFORM benchmark: 16 repeated timeout failures

Measured 16 September 2026 on an AMD Ryzen 9 7950X (32 logical CPUs, 30.5 GiB RAM), with TFORM 5.0.0. The user selected the 16 cases that timed out again in `log_index_20260916_131926_021494.log`. All have one connected gauge sector. Each case ran separately through t^{18} using the production degree-bounded generator, `tform-w32`, and a 600-second expansion limit.

GNU time measured startup, expansion and complete text output to a local file. Compression happened afterward. These times exclude Sage preparation, output parsing, LiE projection, final polynomial construction, cache operations and MySQL writes. No existing database or cache was modified.

Theory ID	Gauge group	TFORM (s)	RSS (GiB)	Retry timeout (s)
20036	$SU(4) \times SU(5)$	24.32	7.19	122.86
20037	$SU(4) \times SU(5)$	24.45	7.19	122.88
20150	$SU(4) \times SU(6)$	23.94	7.19	123.52
20168	$SU(4) \times SU(6)$	24.32	7.19	121.50
20517	$SU(5) \times SU(5)$	37.69	8.84	121.61
20535	$SU(5) \times SU(5)$	38.33	8.84	121.32
20631	$SU(5) \times SU(6)$	37.79	8.84	121.91
20632	$SU(5) \times SU(6)$	37.58	8.84	122.61
20641	$SU(5) \times SU(6)$	24.64	7.18	122.79
20643	$SU(5) \times SU(6)$	24.92	7.19	122.06
20651	$SU(5) \times SU(6)$	25.46	7.26	122.20
20658	$SU(5) \times SU(6)$	25.47	7.25	122.50
20668	$SU(5) \times SU(6)$	24.76	7.18	121.60
20671	$SU(5) \times SU(6)$	24.74	7.18	121.63
22470	$SU(11) \times SU(11)$	37.55	8.84	122.87
22475	$SU(11) \times SU(11)$	37.46	8.83	122.92

All 16 expansions completed successfully: 23.94–38.33 s each, median across theories 25.19 s, total 473.42 s; maximum RSS 8.84 GiB. The original log ran 32 theories concurrently. Its start-to-error intervals are censored and include preparation/scheduling; their quotient with isolated TFORM time is not a controlled FORM speedup.

Success required zero exit status, empty stderr and complete printed output. Two pairs with identical raw programs had identical complete outputs (20517/22475 and 20535/22470). Every large case was not compared against serial FORM, and no complete gauge projection was benchmarked here.

Evidence. `output/benchmarks/tform32_repeated_timeouts_20260916/` contains `report.md`, input/log evidence, machine-readable measurements, the benchmark script, generated programs, hashes and compressed outputs. The script overwrites its own output directory; copy it and change its output path before an independent repeat.

TFORM benchmark: random recovered cases and save-time ratios

The initial run `log_index_20260916_130305_574084.log` had 210 FORM timeouts. Later logs at 13:19:26 and 13:52:04 contained explicit full-index saved messages for 51 of them. A uniform draw without replacement selected 16 before inspecting their complexity:

```
random.Random(3329612950807171883).sample(sorted(eligible_ids), 16)
```

The seed was drawn once from OS randomness. Conditions matched the preceding benchmark: one connected theory at a time, t^{18}, `-w32`, no FORM-cache reuse, one observation per theory, and complete output written to a local file.

ID	Gauge group	TFORM s	Save s	Ratio %	Save/TFORM	GiB
19704	$SU(3) \times SU(4)$	9.00	142.23	6.33	15.80	3.55
19726	$SU(3) \times SU(5)$	14.96	147.70	10.13	9.87	5.49
19739	$SU(3) \times SU(5)$	14.96	76.92	19.45	5.14	5.49
19749	$SU(3) \times SU(5)$	14.90	105.61	14.11	7.09	5.50
19794	$SU(3) \times SU(6)$	9.50	270.95	3.51	28.52	3.65
19920	$SU(4) \times SU(4)$	9.23	251.74	3.67	27.27	3.56
20062	$SU(4) \times SU(5)$	9.48	701.20	1.35	73.97	3.66
20063	$SU(4) \times SU(5)$	9.65	251.11	3.84	26.02	3.66
20109	$SU(4) \times SU(6)$	9.01	304.90	2.96	33.84	3.56
20113	$SU(4) \times SU(6)$	9.17	117.40	7.81	12.80	3.55
20155	$SU(4) \times SU(6)$	9.38	106.25	8.83	11.33	3.65
20230	$SU(4) \times SU(7)$	15.07	148.71	10.13	9.87	5.50
20591	$SU(5) \times SU(6)$	14.95	122.91	12.16	8.22	5.49
20611	$SU(5) \times SU(6)$	9.33	91.99	10.14	9.86	3.61
20620	$SU(5) \times SU(6)$	15.10	180.11	8.38	11.93	5.47
22459	$SU(11) \times SU(11)$	15.22	645.29	2.36	42.40	5.50

Let T_{TFORM} be the isolated expansion wall time and T_{save} the historical interval from calculation start to the explicit full-index saved log line. The report uses unrounded times:

$$r = 100\, T_{\mathrm{TFORM}}/T_{\mathrm{save}}, \qquad q = T_{\mathrm{save}}/T_{\mathrm{TFORM}}.$$

All 16 succeeded: 9.00–15.22 s, total 188.91 s, median across theories 9.575 s; RSS 3.55–5.50 GiB. Here $r = 1.35$–19.45% and $q = 5.14$–73.97. The later save interval includes the whole calculation and database save with 16 or 32 concurrent theories. The table compares different workloads and concurrency conditions; q is not a measured speedup of FORM or the complete index calculation.

Evidence. `output/benchmarks/tform32_recovered_random16_20260916/` retains the selection/population, original log lines, raw programs, output hashes, compressed outputs, measurements and ratio-bearing `report.md`. Neither benchmark was rerun for the 19 September documentation update.

3.3 Gauge singlets, LiE and the reusable character cache

MODULE `index/char_decomposition_cache.py`

The Adams operation is

$$\psi^k \chi_R(g) = \chi_R(g^k), \qquad \prod_{k\geq 1} (\psi^k \chi_R)^{n_k} = \sum_\lambda c_\lambda \chi_\lambda. \tag{3.10}$$

The right-hand side may be a *virtual* character: c_λ need not be nonnegative. For $SU(2)$, $\psi^2 \chi_{\mathbf{2}} = \chi_{\mathbf{3}} - \chi_{\mathbf{1}}$, while $\chi_{\mathbf{2}}^2 = \chi_{\mathbf{3}} + \chi_{\mathbf{1}}$. Replacing the former with the latter would change the PE and corrupt gauge projection. Source: LiE manual, character operations [11]; Sage character operations [2].

A full-decomposition request identifies the Cartan type, Dynkin labels and the complete Adams-power tuple. Its weighted order is

$$w = \sum_{k\geq 1} k n_k. \tag{3.11}$$

This is metadata, not a unique key: $(2,0)$ and $(0,1)$ have the same order but describe different virtual characters. Full decompositions remain available through `get_decomposition`; singlet requests use the smaller calculation below.

Normalized Haar integration and the product-group rule are unchanged:

$$\int_{G_a} \chi_\lambda(g_a)\, d\mu_a = \delta_{\lambda,0}, \qquad \mathrm{mult}_{\mathbf{1},G}(\boxtimes_a V_a) = \prod_a \mathrm{mult}_{\mathbf{1},G_a}(V_a). \tag{3.12}$$

`_project_terms` groups formal products by simple gauge factor and requests one integer per distinct product. It multiplies these integers into the FORM coefficient. Empty products contribute one. B_r, D_r continue to mean Spin representations, not a restriction to representations descending to SO quotients.

Why two partial decompositions suffice

For irreducibles, Schur orthogonality and duality imply

$$\int_G \chi_\lambda(g)\overline{\chi_\mu(g)}\, d\mu(g) = \delta_{\lambda\mu}, \qquad \chi_{\mu^*}(g) = \overline{\chi_\mu(g)},$$
$$[\mathbf{1}](AB) = \sum_\lambda a_\lambda b_{\lambda^*}, \quad A = \sum_\lambda a_\lambda \chi_\lambda, \quad B = \sum_\mu b_\mu \chi_\mu.$$

Equivalently, $(V \otimes W)^G \simeq \mathrm{Hom}_G(V^*, W)$; Hom_G is the vector space of linear maps commuting with the group action. Schur's lemma gives one invariant for dual irreps and zero otherwise. Linearity extends the pairing to signed virtual characters. Source: Etingof, Corollary 35.9 and Proposition 32.1 [22]; Sage `inner_product()` and `dual()` [2]. The displayed contraction follows by substituting the character expansions..

Projection helpers: inputs, outputs and control flow

Helper	Role in the revised algorithm
`_dual_permutation(algebra)`	Cache the permutation of fundamental weights induced by $-w_0$. Apply it to general labels without constructing a new Sage character for each output irrep.
`_singlet_pair(algebra,left,right)`	Return $\sum a_\lambda b_{\lambda^*}$ by looking up dual labels; iterate over the smaller dictionary. Zero and scalar-trivial factors have direct shortcuts.
`_split_character_product`	Expand powers into individual Adams factors, then greedily balance two symbolic partial products.
`_character_singlets`	Plan needed partial decompositions, prefetch same-base-irrep products, expand mixed parts and contract the two sides.
`_tensor_decompositions`	Ask LiE for a full *intermediate* tensor product; never perform the final pairing this way.
`_compute_singlet_multiplicities`	Handle already-decomposed inputs: sort by dictionary size, recursively build two parts, and pair dual irreps.

An input factor `(labels,(n1,n2,...))` means $\prod_j \psi^j(R)^{n_j}$. The splitter assigns each individual $\psi^j(R)$ the estimated cost $j \max(1, \sum_i \lambda_i)$. It sorts largest first and puts the next factor on the lighter side; a tie favors an existing copy of the same base irrep. Repeated factors may separate, while a single Adams operation stays intact. This is a project heuristic, not an optimality theorem.

Inside `_character_singlets`, `plan()` recursively records the required mixed splits and collects requests involving only one base irrep. The existing `get_decompositions()` cache and process pool supply these requests. `expand()` then builds mixed parts with LiE and reuses them in the thread-local `partial_products` dictionary. Only the complete product's final integer is persisted in the singlet table; mixed partial decompositions are memory-only. Trivial base irreps are removed from the calculation because $\psi^j(\mathbf{1}) = \mathbf{1}$.

> **Duality is not conjugate canonicalization**
>
> For A_2, $(a,b)^* = (b,a)$; for A_r, the label order reverses. The general permutation is obtained from the existing Sage duality helper and cached. Actual conjugate orientations remain distinct in character keys. For example, $[\mathbf{1}](\mathbf{3} \otimes \overline{\mathbf{3}}) = 1$, whereas $[\mathbf{1}](\mathbf{3} \otimes \mathbf{3}) = 0$ for $SU(3)$. A pseudoreal irrep pairs with itself with coefficient $+1$; its Frobenius–Schur sign is not inserted into this tensor pairing.

For four $SU(2)$ doublets, decompose the two pairs as $\mathbf{2} \otimes \mathbf{2} = \mathbf{1} \oplus \mathbf{3}$. The final singlet is $1 \cdot 1 + 1 \cdot 1 = 2$, without decomposing the complete four-factor product. The final lookup costs $O(r \min(N_A, N_B))$ label operations for rank r and support sizes N_A, N_B; constructing the partial decompositions can still dominate.

SQLite schema, exact keys and cache lifetime

The character and singlet tables use `sqlite3` and default to `<project-root>/char_decomposition_cache.db`. The independent cache schema has `PRAGMAuser_version=1`; it is unrelated to MySQL theory schema version 1.

Table	Columns and primary key
`character_decompositions`	`algebra TEXT`, `dynkin_labels TEXT`, `adams_powers TEXT`, `adams_order INTEGER`, `terms_json TEXT`. Primary key: `(algebra,dynkin_labels,adams_powers)`.
`singlet_coefficients`	`algebra TEXT`, `product_key TEXT`, `coefficient TEXT`. Primary key: `(algebra,product_key)`.

All columns are `NOT NULL`; both tables use `WITHOUT ROWID`. Dynkin labels and canonical powers are compact JSON text. Coefficients are signed decimal strings, preserving integers beyond SQLite's 64-bit integer storage range [24]. A decomposition payload is a list of `[labels,"coefficient"]` pairs, not a separate JSON cache file.

`_canonical_product` merges repeated labels by adding Adams exponents and sorts factors in descending label order. A symbolic key has the shape `["characters",canonical_product]`. Already-decomposed inputs use a distinct `["decompositions",sorted_encoded_factors]` namespace. The algebra is always part of the SQL key. Product gauge groups reuse simple-factor entries.

`get_singlet_multiplicities()` checks memory and then SQLite before requesting any decomposition. On a miss it invokes `_character_singlets()`, saves the integer and returns results in input order. Cached zero and negative values are valid hits. `get_decomposition()` returns a copy of a stored dictionary; `get_decompositions()` batches missing prerequisites by tensor-factor count. Their full-decomposition algorithm remains available to explicit callers.

Concurrency and failure boundaries

Each thread owns its connection and memory caches. A PID check avoids using an inherited connection; the process-pool parent closes its connection before starting workers, then writes completed results by dependency level. Sequential batches flush at 64 entries and at completion. LiE calculations do not run inside write transactions. Failed calculations do not insert a guessed zero.

WAL permits readers alongside a writer, but still serializes writers; it assumes a local filesystem [23]. The code sets a 30-second busy timeout and retries busy/locked operations. Python's connection context manager controls commit/rollback; this class's `close()` and context manager also close the connection and clear local caches [25].

For `CharacterDecompositionCache`, `cache_directory=` selects a directory for the default filename and `database_path=` selects an explicit file; they are mutually exclusive. Its builder CLI exposes `--cache-directory` and `--cache-database`. Index calculation instead accepts the single file argument `char_cache_database_path=` or CLI option `--char-cache-database`. Legacy JSON discovery/import and `cache_path()` have been removed. Existing SQLite entries and keys remain valid; the projection update needs no migration.

Persisting the FORM expansion before projection

`index/form_expansion_cache.py` now owns `IndexFormTerm`, the exact parser, and `FormExpansionCache`. For raw program text P, the stored value is the parsed list of monomials before any gauge projection:

$$P \longmapsto \{(c, \alpha, \beta, \gamma, ((p, \mathbf{n}_p))_p)\}\,, \qquad c\, t^{\alpha} y^{\beta} u^{\gamma} \prod_{p,j} C_p(j)^{n_{p,j}}.$$

This notation is a direct description of the project record format. Each `IndexFormTerm` has a `Fraction` coefficient, integer fugacity powers, and a tuple of character indices and Adams-power tuples. The compact JSON row is `["numerator","denominator",t_power,y_power,u_power,characters]`. `_encode_expansion` stores a list of these rows; `_decode_expansion` restores exact arbitrary-size fractions and nested tuples. No float conversion or representation decomposition occurs in this layer.

```
CREATE TABLE form_expansions (
    program TEXT COLLATE BINARY NOT NULL PRIMARY KEY,
    expansion_json TEXT NOT NULL
) WITHOUT ROWID;
```

The key is the complete program text, compared verbatim, not a hash. Whitespace, numeric multiplicities, symbol names and the truncation order matter. There is no program normalization or symbolic-multiplicity template. The default file is `form_expansion_cache.db`, separate from the character database; the FORM module does not set a separate `user_version`.

Lookup. `get_expansion(program)` reads and decodes a hit. A miss runs `run_form`, calls `parse_form_output`, encodes the terms and commits with `ONCONFLICT(program)DONOTHING`. Failed execution or parsing does not create an entry. Each thread opens its own connection, with a PID check before reusing it, WAL, a 30-second busy timeout and up to three attempts for busy/locked writes. FORM runs outside write transactions. Concurrent identical misses can execute FORM more than once, but retain only one complete row. The context manager closes the current thread's connection. These boundaries follow the implementation and SQLite's serialized-writer model [23, 25].

Index integration. `calculate_index` and `calculate_index_internal` accept `form_cache_database_path=`; the index CLI exposes `--form-cache-database`. Otherwise the FORM database is placed beside the selected character database. A hit bypasses FORM execution and FORM-output parsing, while group-specific singlet projection still runs. Orders below two return the vacuum without external execution or cache access.

Different theories can share this expansion when their raw programs are identical and they use the same FORM database. Group and irrep meanings are supplied afterward by each theory's character basis. Their final indices can therefore differ despite a shared expansion. This reuse applies to the supported non-SU groups too; it does not require a power-sum/Schur integration backend.

Large cache files and WAL lifetime

A several-gigabyte cache is not by itself a reason to replace SQLite [28]. For this implementation, distinguish indexed-key lookup from decoding one large JSON payload, scanning candidate keys for a dependency plan, and waiting for concurrent writes. Those costs need representative measurements; this session did not benchmark a several-gigabyte cache.

In WAL mode, `-wal` holds the write-ahead log and `-shm` coordinates access. Other open connections or an unclean process exit can leave these files after a program finishes. SQLite normally checkpoints and cleans up on the last clean close; their presence alone is not evidence of corruption [29]. Never remove a live WAL manually: committed data may still be there. A cache context manager closes its current thread's connection; that does not close another process's connection. No new checkpoint or worker-finalizer policy was implemented as part of this discussion.

Reusing lower-order character decompositions

Write the cached virtual character as

$$D_R(\mathbf{n}) = \sum_{\lambda} d_\lambda \chi_\lambda = \prod_{j\geq 1} (\psi^j \chi_R)^{n_j}, \qquad q(\mathbf{n}) = \sum_j j n_j.$$

If $\mathbf{a} + \mathbf{b} = \mathbf{n}$ componentwise, multiplication in the representation ring gives

$$D_R(\mathbf{n}) = D_R(\mathbf{a}) D_R(\mathbf{b}), \qquad d_\nu = \sum_{\lambda,\mu} a_\lambda b_\mu N^\nu_{\lambda\mu}, \quad \chi_\lambda \chi_\mu = \sum_\nu N^\nu_{\lambda\mu} \chi_\nu.$$

The identity follows by multiplying the defining Adams products; LiE's `tensor` implements the last decomposition [11, Sec. 4.5]. Signed coefficients remain valid. The two exact base cases are $D_R(\mathbf{e}_1) = \chi_R$ and $D_{\mathbf{1}}(\mathbf{n}) = 1$; neither needs a LiE call.

`CharacterDecompositionCache._decomposition_dependencies` first tries the usual split: remove one factor at the smallest occupied Adams index. For at most two factors, or when both usual dependencies are available, it retains that split. Otherwise it searches lower-weighted-order keys for the *same* Cartan type and Dynkin labels, including thread-local entries and the known first Adams operation. It considers pairs already available whose padded exponent vectors sum to the requested vector.

Among alternative pairs, the score is the product of the two numbers of irreducible terms, with zero cost for a zero or scalar-trivial factor. Ties use the total number of terms and then the vectors themselves. This estimates tensor work; it does not predict the fastest LiE call or find a globally optimal factorization. The batch dependency planner and the actual calculation use the same selection. Existing SQLite keys, schema and thread/process ownership are unchanged.

For example, if $R^{\otimes 2}$ and $(\psi^2 R)^{\otimes 2}$ are stored, their product can be obtained with one tensor operation on those entries. Atomic prerequisites need not be recalculated just to reconstruct a cached composite. On-demand generation still constructs missing prerequisites when no suitable cached split is available. It does not automatically enumerate all lower orders.

Scope. This optimization is separate from the singlet splitter. The singlet path still decomposes two partial products and contracts dual labels; the new dependency choice helps construct the required same-irrep partial products. Intermediate full tensor decompositions can remain expensive even when all individual Adams operations have already been cached.

Building every product through a chosen Adams order

`build_decomposition_cache(cartan_type,dynkin_labels,max_adams_order,...)` in `index/char_decomposition_cache.py` fills the existing decomposition table for one base irrep. At weighted order q it calls `common.math_utils.frobenius_solve(range(1,q+1),q)` to enumerate

$$\mathcal{P}_q = \left\{ (n_1, \ldots, n_q) \in \mathbb{Z}^q_{\geq 0} : \sum_{j=1}^{q} j n_j = q \right\}.$$

Each solution specifies one Adams product. This is also the multiplicity description of an integer partition: $|\mathcal{P}_q| = p(q)$, with counts $1, 2, 3, 5, 7, 11$ at orders one through six (29 rows in total).

Orders run sequentially. The builder reads existing keys at the current order, skips complete rows without decoding their payloads, and divides missing solutions into batches for a persistent `ProcessPoolExecutor` using `spawn`. Batches contain at most 64 requests, with several batches per worker to balance uneven costs. Workers read prerequisites and calculate; the parent commits each returned batch. It submits the next order only after the current one is fully committed. LiE never runs in a write transaction. Completed batches survive a later failure or interruption, so a rerun resumes from the missing entries. Independent builders may duplicate simultaneous misses; uniqueness and short transactions preserve complete stored rows.

Every composite at order q has a split into two strictly lower orders. Consequently both factors are already stored and at most one tensor operation is needed for that request, with scalar/zero shortcuts as usual. For an initially empty cache of a nontrivial irrep, one uninterrupted build needs only one individual Adams calculation per $j = 2, \ldots, M$: $M - 1$ in total. This counts logical Adams calculations, not backend retries or concurrent duplicate builds. The builder fills same-irrep decompositions, not singlet coefficients, mixed-irrep products or the FORM database.

Cutoff for the full index. A letter starts at t^2, and its j-th Adams contribution starts no earlier than t^{2j}. For each base irrep in a fixed gauge factor, its product therefore satisfies

$$\deg_t \;\geq\; 2\sum_j j n_j = 2q \quad\Longrightarrow\quad q \leq \left\lfloor \frac{N}{2} \right\rfloor \quad \text{for an index through } t^N.$$

This is a bound derived from the implemented single-letter functions. Use $M = \lfloor N/2 \rfloor$ for each required adjoint and matter irrep, retaining distinct conjugate labels. It is a sufficient bound, not a claim that every partition will occur in the projector. For product groups it holds factorwise; one bifundamental letter contributes a character to multiple factors, so one must not sum their separate orders and apply the same bound to that sum. Through t^{18}, $M = 9$ suffices. For $N < 2$ there is no cache to prebuild.

Builder API, dependencies and command-line examples

With Sage's environment active, prepare the A_2 fundamental through order nine:

```
sage -python -m index.char_decomposition_cache A2 \
  --dynkin-labels 1 0 --max-adams-order 9 --processes 4 \
  --cache-database /tmp/index-demo/char_decomposition_cache.db
```

Repeat for every required irrep. For A_2 SQCD these include $(0,1)$ and the adjoint $(1,1)$ in addition to $(1,0)$. The group rank does not multiply the maximum Adams order. The direct script entry point `sage-pythonindex/char_decomposition_cache.py...` is also supported.

```
from index.char_decomposition_cache import build_decomposition_cache

if __name__ == "__main__":
    totals = build_decomposition_cache(
        "A2", (1, 0), 9, processes=4,
        database_path="/tmp/index-demo/char_decomposition_cache.db",
        progress=lambda q, total, computed:
            print(q, total, computed),
    )
```

The return value is `dict[int,int]` mapping each order to its total product count, including reused rows. The optional callback receives `(order,total,computed)` in the parent after that order is committed. `processes=1` is serial; the default is the available CPU count. The pool starts only when there is parallel work. Use a `__main__` guard in scripts with multiple processes.

The maximum order and worker count must be positive integers. Cartan type, rank and Dynkin labels are validated. `cache_directory=` and `database_path=` are mutually exclusive. Other keywords are `lie_executable=` and `timeout=` (600 seconds per LiE call by default; Python accepts `None` for no timeout). The CLI provides `--max-order` as an alias for `--max-adams-order`, as well as `--cache-directory`, `--lie-executable`, and numeric `--timeout`. It prints computed/reused counts and exits with status 2 on a reported failure.

The builder imports `frobenius_solve` lazily and therefore needs OR-Tools only when called; ordinary index/cache lookups do not acquire that dependency. Its enumeration and complete decompositions can grow much faster than the subset used by one index. Prebuilding is optional, and a fully warm repeat still enumerates solutions and checks keys even though it runs no LiE or worker pool.

Use the prepared character database and a shared FORM database for index calls:

```
sage -python -m index.n2_theory_index theory.json --order 18 \
  --char-cache-database /tmp/index-demo/char_decomposition_cache.db \
  --form-cache-database /tmp/index-demo/form_expansion_cache.db
```

Without the last option, the FORM database defaults to that same directory. The corresponding index Python keywords are `char_cache_database_path=` and `form_cache_database_path=`.

Earlier sparse-projection measurements, before FORM caching

The temporary candidate was compared against commit `a9b7009` before promotion to `0a402ec`. The following historical measurements predate the FORM cache and cached-factor planner; their warm runs still execute FORM. All seven comparative index cases agreed exactly, including product groups and half hypers. There were 319 matching singlet queries; 202 also matched independent $SU(2)/SU(3)$ weight calculations. All 216 dual-label comparisons across 18 Cartan types matched Sage. All 4,996 old SQLite singlet entries tested were reused without recalculation. These are project measurements, not claims from the mathematical references.

Cold calculation	Previous (s)	Revised (s)	Speedup
A_8, through t^{18}, projection	4.599	0.849	5.42×
A_{16}, through t^{12}, projection	1.473	0.207	7.12×
A_{32}, through t^8, projection	0.469	0.117	4.01×
A_8, through t^{18}, complete index	6.213	2.443	2.54×

These are three-trial medians with fresh SQLite files and one worker for SQCD with $2(r+1)$ full fundamental hypers. Warm A8 complete-index time stayed about 1.70 s. Four-worker comparisons also agreed; small cases can incur process startup overhead. Reproduce with `test/benchmark_character_cache.py`, `--baseline-ref a9b7009`, `--baseline-label existing`, `--candidate-label sparse`.

On 10 September, $A_{32} = SU(33)$ with 66 full fundamental hypers completed through t^{18} with `timeout=None` and one worker: 94.293 s cold, 1.786 s warm. The two results matched exactly and contain 117 nonzero monomials. One LiE calculation of $\psi^3(\mathrm{adj})^2$ took 82.699 s, or 87.7% of the cold total. Each input contains 10 irrep terms; the output contains 91. FORM took 1.357 s, and all other LiE calls together took 5.066 s. This is one profiled run, not a median or an independent proof of every coefficient.

The final pairing avoids the complete tensor decomposition but not every intermediate one: $\psi^3(\mathrm{adj})^3$ splits as $\psi^3(\mathrm{adj})^2$ and $\psi^3(\mathrm{adj})$. That two-factor intermediate is the measured bottleneck. Detailed local evidence is in `output/benchmarks/singlet_projection_benchmark.md` and `output/benchmarks/a32_unlimited/report.md`.

Public result and a regression example

`calculate_index(data,order,...)` returns an exact flat Sage Laurent polynomial in (t, y, u) over $\mathbb{Q}$. `calculate_index_internal` accepts already parsed factors and hypers; `calculate_index_from_file` reads JSON, and `parse_index_polynomial` reverses the serialized polynomial format. For $SU(3)$ with six full fundamentals, the tested expansion is

$$\begin{aligned}\mathcal{I} = 1 &+ (36u^{-2} + u^4)t^4 - u^2(y + y^{-1})t^5 \\ &+ (40u^{-3} - 36 + u^6)t^6 + O(t^7).\end{aligned} \tag{3.13}$$

The displayed $O(t^7)$ is explanatory: the returned Laurent polynomial does not store a precision bound. Cache reuse improves repeated calculations, but the number of terms and tensor decompositions still grows rapidly with the cutoff.

FORM cache measurements with character projection prefilled

The controlled comparison used nine theories at t^{12} and t^{18}, one warmup and five measured repetitions per mode, with fresh clients and one worker. Both decompositions and final singlet coefficients were already filled. The character database was query-only and guards prohibited LiE execution. All 270 timed indices agreed exactly. Medians below are full calculation times in seconds, excluding setup and Python/Sage startup.

Theory, through t^{18}	Disabled	Miss/save	Hit	Speedup
$SU(2)$, 4 fundamentals	0.4323	0.4916	0.0617	7.00×
$SU(3)$, 6 fundamentals	1.7220	1.7959	0.2304	7.48×
$SU(5)$, 10 fundamentals	1.7265	1.7839	0.2648	6.52×
$SU(3)$, 1 adjoint ($\mathcal{N}=4$)	0.2808	0.3180	0.0429	6.54×
$Sp(2)$, 6 fundamentals	0.4434	0.4817	0.0634	7.00×
Spin(8), 6 vectors	0.4329	0.4848	0.0655	6.60×
G_2, 4 fundamentals	0.4418	0.4874	0.0640	6.90×
$SU(2)^2$, 2 bifundamentals	2.7421	2.7843	0.3251	8.44×
$SU(5)$, symmetric + antisymmetric	13.7565	14.1896	1.8771	7.33×

All matter in this table is full hypermultiplets; $Sp(2) = USp(4)$. Disabled mode bypasses FORM SQLite and executes/parses FORM; miss/save begins with an initialized empty FORM database and includes serialization and insertion. Hit mode decodes an existing expansion with FORM execution forbidden. The speedup is disabled time divided by hit time. At t^{12} the range was 3.23–6.92×. These are warm-filesystem measurements on this machine, not cold-disk or contention benchmarks. Evidence: `report.md`, `results.json` and `benchmark.py` under `output/benchmarks/form_expansion_theories/`.

Cross-theory hits. Two pairs share byte-identical programs: $SU(2)$ with four fundamentals and G_2 with four fundamentals; $Sp(2)$ with six fundamentals and Spin(8) with six vectors. Both directions at both orders gave eight passing reuse checks. The first theory executed FORM once; a new client for the second executed/parsed FORM zero times, decoded once, and left the database at one row. Each final index matched its own independent fresh-FORM result, while the paired final indices differed. A changed-program control missed as expected. See `cross_theory_reuse.md`, `cross_theory_reuse.json` and `check_cross_theory_reuse.py` in that same directory.

I/O cost, decomposition planning and builder verification

A separate I/O benchmark used the 113,831-term t^{18} expansion: a 589-byte program and 5,907,568-byte JSON payload (5.91 MB). Thirty measured repetitions followed three warmups, using SQLite WAL with synchronous FULL on the project filesystem. Medians are milliseconds:

Operation	Median (ms)
Serialize exact terms	179.32
SQLite insert and commit	42.07
Total save, excluding database initialization	221.51
SQLite read and fetch JSON	3.08
Reconstruct `IndexFormTerm` objects	866.42
Hit with an already open connection	870.81
Hit with a new client, including close	894.65

Initialization separately took 31.43 ms. No FORM execution/parsing is included; every round trip was checked exactly. Python reconstruction dominated loading. Individual component medians need not add to the median total. This is a single-process warm-filesystem test, not concurrent lock contention. Evidence: `output/benchmarks/form_expansion_cache_io/report.md`, `timings.json` and `benchmark.py`.

Cached-factor planner. The temporary candidate matched 130 products over A_1, A_2 fundamental/adjoint, C_2 and G_2 in serial and three-worker runs; $SU(2)$ cases also matched an independent weight oracle. Prepared-cache misses improved 1.31–1.72× for the single-request API and 1.03–1.17× for the batch API, reducing two tensor calls to one. A case with only composite entries also reduced one Adams call to zero. However, 72 full-index comparisons over six theories, with FORM prefilled and either a cold character cache or a cache filled through t^{12}, showed *no consistent overall speedup*. All indices agreed and tensor-call counts were unchanged. The retained baseline, candidate, report, scripts and raw measurements are in `output/benchmarks/adams_cache_planner/`.

Complete builder. Twelve regressions cover all 29 $SU(2)$ products through order six against independent weights, serial/parallel agreement for A_1, A_2, C_2, G_2 through order five, pool reuse, warm skips, partial-cache resume, failure persistence, input validation and both CLI entry points. A real worker log records exactly one Adams call at each of orders two through six and 23 tensor calls, with multiple worker PIDs. This verifies the operation-count claim on the tested build; no broad prebuild speedup has been measured.

The full suite rerun during the 10 September 2026 cache-documentation update ran **209 tests: 207 passed and two live-MySQL tests skipped**. That run used actual Sage/FORM/LiE for backend tests; its MySQL migration and insertion unit tests used mocks unless a test server was explicitly configured. Schema migration tests were later removed when the production layout was reset to schema 1. The new modules are `test_form_expansion_cache.py`, `test_cached_decomposition_planning.py` and `test_build_decomposition_cache.py`. These measurements and algorithms are source-derived project evidence, not results attributed to external papers.

3.4 Disconnected gauge sectors and the factorization identity

A product gauge group is not necessarily a product of independent theories: matter charged under several factors couples those factors. For each nonzero hypermultiplet h, define its gauge support

$$S_h = \{a : \ R_{h,a} \neq \mathbf{1}\}.$$

`split_disconnected_sectors` builds an undirected NetworkX graph with one vertex per gauge factor. For a support $(a_1, \ldots, a_r)$, it inserts the $r-1$ edges $(a_1, a_2), \ldots, (a_1, a_r)$. A star joins every vertex of that support by a path; a complete clique would give the same connected components. Overlapping supports join transitively. Zero-multiplicity matter is ignored and isolated gauge vertices are retained. Connected components are obtained with `networkx.connected_components` [27].

For components $C_1, \ldots, C_s$, each charged hyper belongs to exactly one component. The code preserves factor order, restricts representation and beta contribution dictionaries, and converts a one-factor product hyper into `HyperData`. Multi-factor components retain `ProductHyperData`. No discarded spectator singlet contributes an extra dimension factor.

Public API. `split_disconnected_sectors(factors,hypermultiplets)` returns a list of pairs: a tuple of `GaugeFactorData` and a list of `HyperData` or `ProductHyperData`. Input factor IDs and order are retained. The basic property calculator materializes these IDs as a tuple of tuples, not lazy iterators; it returns `None` for a non-SCFT candidate. The free sector contributes an empty tuple, serialized as an empty JSON array. Database ownership of those IDs is described in the implementation reference.

Let L_α denote the letters of component α and L_0 all matter with empty gauge support. The exponent splits because Adams substitution and summation are linear, so the plethystic identity gives

$$L = L_0 + \sum_\alpha L_\alpha, \qquad \mathrm{PE}[L] = \mathrm{PE}[L_0] \prod_\alpha \mathrm{PE}[L_\alpha].$$

Normalized Haar integration also factorizes, since each L_α depends only on $g_\alpha \in G_{C_\alpha}$:

$$\begin{aligned} \mathcal{I} &= \int \prod_\alpha d\mu_\alpha(g_\alpha)\ \mathrm{PE}[L_0] \prod_\alpha \mathrm{PE}[L_\alpha(g_\alpha)] \\ &= \mathcal{I}_0 \prod_\alpha \mathcal{I}_\alpha, \qquad \mathcal{I}_0 = \mathrm{PE}[L_0], \quad \mathcal{I}_\alpha = \int d\mu_\alpha\ \mathrm{PE}[L_\alpha]. \end{aligned}$$

This follows directly from the plethystic definition and product measure [10, 9]; the graph is the implementation's test for these independent integration variables.

Why all-gauge-singlet matter appears once. A gauge-singlet hyper has empty support. Dropping it loses a physical free-matter factor. Copying it into each of s components would replace $\mathcal{I}_0$ by $\mathcal{I}_0^s$. The code puts all such hypers in one additional sector with no gauge factors, preserving each original multiplicity. "Once" means one assignment of each input species, not changing its multiplicity to one.

3.5 Exact multiplication, flavor characters and database reuse

At finite cutoff, multiplication and truncation commute in the needed sense:

$$\left[\prod_{\alpha=0}^{s}\mathcal{I}_\alpha\right]_N=\left[\prod_{\alpha=0}^{s}[\mathcal{I}_\alpha]_N\right]_N.$$

All sector indices have nonnegative t degree. Discarded powers above N cannot multiply later terms back down to N. Laurent powers of y,u do not alter that argument. `_truncate_index` filters the exact Sage monomial dictionary inclusively after each sector and each product. An identical sector within the same call is looked up or calculated once, but its index is still multiplied once per occurrence.

Flavor refinement. The same identity holds with external flavor variables x: $\mathcal{I}(t,y,u;x)=\mathcal{I}_0(t,y,u;x)\prod_\alpha\mathcal{I}_\alpha(t,y,u;x)$. Shared flavor variables are coefficients, not integrated gauge variables. Products of their characters describe tensor products and need not be decomposed first. For example, a free full hyper with $U(1)$ fugacity x contributes $\mathrm{PE}[f_H(x+x^{-1})]$ once. Gauging a shared flavor subgroup introduces a common Haar integral and can connect sectors; one cannot then multiply separately projected indices. The current code is flavor-unrefined: this is the extension's mathematical contract, not an implemented flavor API.

Optional theory database. `calculate_index` and `calculate_index_internal` accept `theory_db_connection=None`. For each gauged sector, `common.n2_theory_db.find_superconformal_index` matches the existing canonical Lagrangian hash and reads an index only if it is a JSON string with recorded `superconformal_index_order>=order`. Canonicalization retains the established full/half and conjugation conventions; factor IDs are ignored but input factor order remains significant. Missing/null indices and unknown/lower cutoffs are misses. A malformed index or one with negative t powers is rejected by the caller and recalculated. The cutoff is never inferred from the highest surviving monomial.

Misses use `_calculate_sector_index`: character basis, degree-bounded FORM expansion/cache and the existing LiE singlet projection. Free sectors bypass the theory lookup. No sector theories or indices are inserted. A supplied connection is borrowed read-only and is never committed, rolled back or closed by this calculation; it must belong to the calling process/thread and already have the current schema. GUI/database workers pass their own connection. Without one, sector splitting and multiplication still apply.

The character-cache file is a distinct parameter: `char_cache_database_path=None`. It replaces `cache_directory` and `database_path` in the index API; the index and database-worker CLI use `--char-cache-database`. The independent FORM file uses `form_cache_database_path`. The standalone character-cache class/builder retain their own directory/file API and CLI. Saved GUI key `cache/character_database` is unchanged.

3.6 Product-group bottlenecks and recorded evidence

The 14 September investigation found 39 full-index FORM timeouts in a 32-worker run through t^{18}: 29 inputs had disconnected gauge sectors. The main isolated cost was constructing the joint formal-character exponential, before Python parsing, LiE projection or database writes. A printing-disabled control for disconnected $SU(3) \times SU(3)$ theory 9780 still took 516.16 s and retained 2,649,141 formal terms, whereas the final projected index has 168 monomials. This locates that delay in symbolic computation, not an observed database deadlock. Large expansions also incur parsing and memory costs. The report's saved timeout setting was 60 s; the code default was 600 s. Those are dated observations, not assertions about the user's current settings.

Implementation	Effect and evidence
Disconnected-sector split	Projects small independent sectors before multiplying their final polynomials. On 15 September, theory 9780 took 13.753 s with one unique sector calculation and exact agreement with all 168 saved combined-result monomials. Character cache was a private populated copy; FORM cache was empty; no MySQL connection was used.
Degree-bounded FORM	Helps connected sectors too, by omitting products above the remaining degree budget. The 16 September matched bifundamental run is recorded in the FORM explanation.
Remaining limits	Connected sectors can still have many formal monomials and expensive LiE intermediates. Concurrent misses can still duplicate one FORM program; no in-progress job claim or streaming projection was added.

Evidence files. `output/benchmarks/product_form_20260914/report.md` retains the original profiling controls and inputs; `output/benchmarks/disconnected_sectors_20260915.json` records the implemented split. The earlier diagnostic degree-bounded variant became the production generator on 16 September. Historical timings with different printing or cache conditions are not interchangeable benchmarks.

Regression coverage. `test/test_disconnected_index.py` tests transitive and trifundamental connectivity, isolated vectors, singlet matter, zero multiplicities, factor order, repeated sectors and exact comparison with an unsplit calculation. Database cases cover missing/null indices, unknown, lower, equal and higher cutoffs, canonical identity and borrowed-connection ownership. The recorded 15 September combined suite passed 353 tests in 58.615 s with no skips. The subsequent cache-argument revision passed 355 tests in 60.129 s. These are implementation-time results, not reruns performed for this document update.

3.7 Coulomb branch: invariant degrees and operator counting

MODULE `index/n2_theory_coulomb_branches.py`

The old module name `n2_theory_branches.py` has been replaced by the name above. A scalar Coulomb primary obeys $j_1 = j_2 = R = 0$ and $E = -r = \Delta$ in (3.1). Its weight is $(t^2u^2)^\Delta$. For the ordinary scalar Coulomb chiral ring, let

$$\mathcal{I}_C(x) = \sum_{\Delta \geq 0} \dim(\mathcal{R}_C)_\Delta \, x^\Delta, \qquad x = t^2u^2. \tag{3.14}$$

For a Lagrangian semisimple gauge theory, the adjoint scalar ϕ has dimension one and the invariant ring is

$$\mathbb{C}[\mathfrak{g}_\mathbb{C}]^{G_\mathbb{C}} \simeq \mathbb{C}[\mathfrak{h}_\mathbb{C}]^W = \mathbb{C}[P_1, \ldots, P_\ell], \qquad \Delta(P_i) = d_i = m_i + 1. \tag{3.15}$$

Here $\ell = \operatorname{rank} G$, d_i are Weyl invariant degrees and m_i are Weyl exponents. Hence

$$\mathcal{I}_C(x) = \mathrm{PE}\left[\sum_i x^{d_i}\right] = \prod_i \frac{1}{1 - x^{d_i}}. \tag{3.16}$$

Source: Invariant-degree interpretation and tables in Sage [14]; Casimir operators and Coulomb dimensions in Shapere–Tachikawa, §3 and §5.1 [15]; Coulomb-index letters in [16], Eq. (4.17).

Type	Basic invariant degrees (with multiplicity)
A_r	$2, 3, \ldots, r+1$
B_r, C_r	$2, 4, \ldots, 2r$
D_r	$2, 4, \ldots, 2r-2$ together with r
E_6	$2, 5, 6, 8, 9, 12$
E_7	$2, 6, 8, 10, 12, 14, 18$
E_8	$2, 8, 12, 14, 18, 20, 24, 30$
F_4	$2, 6, 8, 12$
G_2	$2, 6$

`_invariant_degrees` caches `WeylGroup(cartan_type).degrees()`. `coulomb_branch_spectrum_from_gauge_factors` accepts one `GaugeFactorData` or an iterable and returns the sorted integer tuple for all factors. Multiplicities are preserved: D_4 gives $(2, 4, 4, 6)$, not $(2, 4, 6)$. Matter information is not needed for this invariant-ring computation. The group-only function does not independently test whether matter makes the theory conformal.

3.8 Coulomb PE with rational dimensions and FORM

The spectrum interface also accepts positive exact rational dimensions, so it can serve as a counting backend for future non-Lagrangian inputs. If $f(x) = \sum_\Delta c_\Delta x^\Delta$, then

$$\mathrm{PE}[f] = \exp\left(\sum_{k\geq 1} \frac{f(x^k)}{k}\right) = \prod_\Delta (1-x^\Delta)^{-c_\Delta}. \tag{3.17}$$

The spectrum API requires $c_\Delta \in \mathbb{Z}_{\geq 0}$; the lower-level `calculate_plethystic_exponential` accepts signed integer coefficients. Negative entries give numerator factors and can describe complete-intersection relations, but an arbitrary signed list is not automatically a valid ring. Source: Plethystic exponential and product identity, [10], §5.

Clearing denominators exactly

For inclusive physical cutoff D, the implementation sets

$$L = \mathrm{lcm}(\mathrm{den}(D), \{\mathrm{den}(\Delta) : c_\Delta \neq 0,\ \Delta \leq D\}), \quad q = x^{1/L}, \quad M = LD. \tag{3.18}$$

Every input monomial becomes $q^{L\Delta}$, with integer exponent. If the lowest nonzero exponent is ν, the exact finite computation is

$$A_M(q) = \sum_{k=1}^{\lfloor M/\nu \rfloor} \frac{1}{k} \sum_\Delta c_\Delta q^{kL\Delta}, \qquad [\mathrm{PE}(f)]_{q\leq M} = \left[\sum_{j=0}^{\lfloor M/\nu \rfloor} \frac{A_M(q)^j}{j!}\right]_{q\leq M}. \tag{3.19}$$

`_build_plethystic_form_program` uses FORM's bounded q symbol and the exponential recurrence (3.9). `_parse_form_series` reads exact rational coefficients; `_to_coulomb_series` maps q^m back to $x^{m/L}$ in `PuiseuxSeriesRing(QQ,"x")`. Empty retained input returns one without starting FORM. The scaling formulas are derived directly from the source; no floating-point fit to exponents is used.

Two useful examples

For $SU(3)$, independent generators have dimensions two and three:

$$\begin{aligned}\mathcal{I}_{\mathrm{C}} &= (1-x^2)^{-1}(1-x^3)^{-1} \\ &= 1 + x^2 + x^3 + x^4 + x^5 + 2x^6 + x^7 + 2x^8 + O(x^9).\end{aligned} \tag{3.20}$$

The two terms at degree six are products P_2^3 and P_3^2; neither is a new generator. With one input dimension $6/5$ and cutoff $18/5$, the result is $1 + x^{6/5} + x^{12/5} + x^{18/5}$.

> **Mathematical input is not physical classification**
>
> Positive rational dimensions are accepted algebraically, including values that would violate the scalar unitarity bound in a unitary four-dimensional SCFT. The code does not establish that a supplied non-Lagrangian spectrum is realized by a theory or that its chiral ring is freely generated.

3.9 Taking the Coulomb limit of an existing full index

`calculate_coulomb_branch_index_from_full_index` accepts the project's Sage Laurent polynomial or its serialized string. Substitute

$$t \longrightarrow 0, \qquad u = \frac{x^{1/2}}{t}, \qquad y \text{ fixed}; \qquad t^a y^b u^c = t^{a-c} y^b x^{c/2}. \tag{3.21}$$

Thus a canonical nonzero monomial has the following disposition:

Condition	Implementation
$a > c$	Vanishes; discard it.
$a = c,\ b = 0,\ c \geq 0$	Keep its coefficient at dimension $\Delta = c/2$.
$a < c$	Raise a divergent-limit error.
$a = c,\ b \neq 0$	Reject a surviving spin-dependent term.
$a = c,\ c < 0$	Reject a negative Coulomb dimension.

An optional `max_dimension` further truncates the retained terms. Like terms are already combined in the source Laurent ring. This path is coefficient selection in Sage/Python and does not run FORM or LiE; symbolic PE/PL is where FORM is used. Source: Project-variable derivation from (3.2); in the same limit (3.3) and (3.4) become $f_V = x$, $f_H = 0$, matching [16], Eq. (4.17).

Precision cannot be created by taking a limit

If the original index is known through t^N (inclusive), then x^Δ comes from $t^{2\Delta}u^{2\Delta}$. Guaranteed Coulomb information therefore extends only to

$$\Delta \leq N/2. \tag{3.22}$$

An order-18 full index can determine Coulomb coefficients through dimension nine, not ninety. The separate index workflow obtains the order-90 Coulomb index directly from Weyl degrees instead. From (3.13), the supported Coulomb result is $1 + x^2 + x^3$ through dimension three.

> **Caller-owned cutoff and adapter restrictions**
>
> The full-index ring and the returned finite Coulomb expressions do not carry the provenance of a truncated polynomial. The caller must retain the trusted cutoff and must not interpret omitted coefficients above it as zero. The full index adapter accepts integer Laurent exponents in t, y, u; it is not a generic adapter for arbitrary fractional-power non-Lagrangian full indices. Direct Coulomb spectrum/series input supports rational dimensions more generally.

For non-Lagrangian applications this adapter also assumes the desired limit is the ordinary scalar Coulomb ring. More general protected contributions are not identified or decomposed by the code. Rejection of spin-dependent terms is a guard, not a proof of all the required physical assumptions.

3.10 Plethystic logarithm: recovering generator dimensions

For $H(0) = 1$, the inverse plethystic transform is

$$\mathrm{PL}[H](x) = \sum_{k\geq 1} \frac{\mu(k)}{k} \log H(x^k), \qquad \mathrm{PL}[\mathrm{PE}(f)] = f. \tag{3.23}$$

$\mu(1) = 1$; $\mu(k) = 0$ when k contains a squared prime factor. Otherwise $\mu(k) = (-1)^s$, where s is the number of distinct prime factors of k. Source: Plethystic logarithm/Möbius inversion, [10], §5.

`calculate_plethystic_logarithm` normalizes the input to a finite exact series, requires its constant coefficient to be one, rejects negative powers, checks any available finite-series precision, and rescales by (3.18). For input $H + O(x^P)$, the inclusive cutoff must satisfy $D < P$; a plain polynomial cannot enforce this provenance check. Write the resulting input as $1 + \delta(q)$ with lowest nonzero power ν and $K = \lfloor M/\nu \rfloor$. FORM evaluates

$$[\log(1+\delta)]_{q\leq M} = \left[\sum_{j=1}^{K} \frac{(-1)^{j+1}}{j}\delta^j\right]_{q\leq M}. \tag{3.24}$$

`_build_plethystic_log_form_program` starts with $z\delta$ and uses

$$z \longmapsto 1 - \frac{j-1}{j} z\delta, \qquad j = 2, \ldots, K. \tag{3.25}$$

After step j, this gives the completed terms up to δ^{j-1} plus $z(-1)^{j+1}\delta^j/j$; setting $z = 1$ proves the recurrence.

If FORM returns $\log H(q) = \sum_m b_m q^m$, Python then applies the exact Möbius coefficient transform

$$[q^n]\,\mathrm{PL}(H) = \sum_{k|n} \frac{\mu(k)}{k} b_{n/k}. \tag{3.26}$$

The helper names are `_mobius` and `_apply_mobius_transform`. The returned Puiseux series contains signed rational coefficients; FORM performs the ordinary-log polynomial algebra, while Python handles this arithmetic transform.

What the extraction wrapper guarantees

`extract_coulomb_branch_spectrum_from_index` expands the PL and returns a sorted tuple containing each dimension as many times as its coefficient. It requires every retained coefficient to be a nonnegative integer; otherwise it raises an error. Use the raw PL routine when signed data is required. For (3.20), the PL through dimension eight is $x^2 + x^3$ and the tuple is $(2, 3)$. For $(1 - x^4)/(1 - x^2)^2$, the PL is $2x^2 - x^4$; the raw result retains the relation and the free-spectrum wrapper rejects it.

> **A finite nonnegative PL is not a global freeness proof**
>
> Generators above the cutoff and relations first appearing later are invisible. For non-complete-intersection rings, higher PL terms can encode syzygies and their cancellations; positive terms are not universally all minimal generators. The wrapper produces a free-generator interpretation only through the supplied trusted cutoff, under the stated ring assumptions.

3.11 Coulomb API guide and reuse boundaries

All names below are in `index/n2_theory_coulomb_branches.py`.

Entry point	Input and result
`coulomb_branch_spectrum_from_gauge_factors`	One/iterable of `GaugeFactorData`; integer dimension tuple. Uses Sage degrees only.
`calculate_lagrangian_coulomb_branch_index`	Gauge-factor data and cutoff; Weyl spectrum → FORM PE → exact x series.
`calculate_coulomb_branch_index`	Exactly one of spectrum or `full_index`. Spectrum requires a cutoff; full-index mode permits it.
`calculate_coulomb_branch_index_from_full_index`	Laurent polynomial/string; select terms by (3.21).
`calculate_plethystic_exponential`	Positive rational dimensions mapped to signed integer coefficients, plus cutoff.
`calculate_plethystic_logarithm`	Unit-constant Coulomb series/string and trusted cutoff; signed rational Puiseux series.
`extract_coulomb_branch_spectrum_from_index`	Same index input; repeated `Fraction` dimensions if the retained PL is non-negative integral.
`parse_coulomb_branch_index`	Arithmetic-only serialized x expression; exact Sage Puiseux expression.

Minimal example, run with Sage Python

```
from fractions import Fraction
from anomalies.check_n2_anomalies import GaugeFactorData
from anomalies.lie_algebra import get_lie_algebra
from index import n2_theory_coulomb_branches as cb

gauge = GaugeFactorData("g", get_lie_algebra("A2"))
H = cb.calculate_lagrangian_coulomb_branch_index(gauge, 8)
dims = cb.extract_coulomb_branch_spectrum_from_index(H, 8)
assert dims == (Fraction(2), Fraction(3))

H_fractional = cb.calculate_coulomb_branch_index(
    [Fraction(6, 5)], Fraction(18, 5)
)
```

String parsers permit only the documented arithmetic variables and syntax, not general Sage code. A textual $O(x^D)$ precision annotation is not the serialized polynomial format. Supply exact fractions, not float approximations such as `1.2`, and retain the cutoff separately when serializing a truncated result.

The common property module has similarly named functions taking a *theory dictionary*, not `GaugeFactorData`. Use explicit imports to distinguish the two API layers. Its Lagrangian spectrum path reads Weyl degrees directly; it does not need to compute a PL of its already-known spectrum.

3.12 Future extension: Hall–Littlewood and Higgs operators

> **Mathematical design notes only**
> There is currently no Higgs/HL calculation module, flavor-refined index output, or Higgs-spectrum database property. This section records the mathematical background and proposed implementation changes from the session, not existing functionality.

Higgs-branch chiral primaries are scalar $\widehat{\mathcal{B}}_R$ primaries with

$$E = 2R, \qquad r = 0, \qquad j_1 = j_2 = 0. \tag{3.27}$$

The HL trace instead imposes $\delta_R = 0$ and $\delta_{1\pm} = E \pm 2j_1 - 2R - r = 0$. Solving these equations gives

$$j_1 = 0, \qquad j_2 = r, \qquad E = 2R + r, \qquad \mathcal{I}_{\mathrm{HL}} = \mathrm{Tr}_{\mathrm{HL}}(-1)^F \tau^{2(E-R)}. \tag{3.28}$$

These are conditions on contributing states, including possible fermionic cohomology representatives, not only Higgs scalar primaries. On (3.27), the exponent becomes $E = 2R$. This explains why the two sets of conditions differ without contradicting one another. Source: Gadde–Rastelli–Razamat–Yan, Eqs. (4.1), (4.5)–(4.8) and Appendix B [16]; general caveats about identifying HL with a Higgs Hilbert series in [19].

The limit in the project's variables

Keeping $\tau = t^2/u$ fixed and sending $t \to 0$ gives

$$u = t^2/\tau, \qquad t^a y^b u^c = t^{a+2c} y^b \tau^{-c}, \qquad f_H \longrightarrow \tau, \quad f_V \longrightarrow -\tau^2. \tag{3.29}$$

Thus $a + 2c = 0$ selects surviving terms, positive values vanish, and negative values signal a divergent limit. Surviving y dependence should cancel for an HL result. A full index known to t^N supports integer HL degree at most $\lfloor N/2 \rfloor$. Direct HL construction would instead use the degree-one hyper letter, so its Adams/particle bound is D, not the current full-index $\lfloor D/2 \rfloor$.

Why the vector letter represents quadratic constraints

For full hypers, the $\mathcal{N} = 2$ superpotential can be written

$$W = \sqrt{2} \sum_a \Phi_a^A \mu_a^A(Q, \widetilde{Q}), \qquad \mu_a^A = \sum_h \widetilde{Q}_h T^A_{R_{ha}} Q_h. \tag{3.30}$$

At $\Phi = 0$, the nontrivial F-term equations are $\partial W / \partial \Phi_a^A = \sqrt{2}\mu_a^A = 0$. They are quadratic in hyper scalars and transform in adj_a; derivatives with respect to the hypers vanish there. Half-hyper notation uses the invariant symplectic pairing and gives the same quadratic moment-map structure. Source: Superpotential/F-term argument in [16], Eqs. (4.9)–(4.11); general Lagrangian background in [17].

3.13 Future extension: constraints, syzygies and flavor refinement

Let $S = \mathbb{C}[Q, \widetilde{Q}]$ denote the polynomial ring of complex hyper coordinates. The Higgs chiral ring in the chosen complex structure is

$$\mathcal{R}_H = \left(S/(\mu_a^A)\right)^{G_{\mathbb{C}}}. \tag{3.31}$$

If the moment-map components form a homogeneous regular sequence, quotienting by them multiplies the equivariant Hilbert series by

$$\mathrm{PE}\left[-\tau^2 \sum_a \chi_{\mathrm{adj}_a}(g_a)\right], \qquad H_H(\tau) = \int_G d\mu\ \mathrm{PE}\left[\tau\chi_{\mathrm{hyp}} - \tau^2\chi_{\mathrm{adj}}\right]. \tag{3.32}$$

For one homogeneous non-zero-divisor f of degree d, the exact sequence $0 \to S(-d) \xrightarrow{\times f} S \to S/(f) \to 0$ gives the factor $1 - \tau^d$. Iterating is valid for a regular sequence. More generally, the fermionic factor is the Euler characteristic of the Koszul complex with odd variables η_a^A and differential $d\eta_a^A = \mu_a^A$. Its zeroth homology is the quotient; higher homology can obstruct equality with the HL index. Source: Regular/Koszul sequences [20]; physical HL/Higgs caveat [19].

For $SU(2)$, explicitly,

$$\mathrm{PE}[-\tau^2\chi_{\mathbf{3}}(a)] = (1-\tau^2a^2)(1-\tau^2)(1-\tau^2a^{-2}) = 1 - \tau^2\chi_{\mathbf{3}} + \tau^4\chi_{\mathbf{3}} - \tau^6. \tag{3.33}$$

The higher terms are the alternating exterior powers of the constraint space. One must check assumptions for the theory in question; a blanket statement that every HL index, or every genus-zero construction, equals a Higgs Hilbert series is too strong [19].

What flavor-unrefinement loses

A refined Hilbert series organizes graded operators as

$$H_H(\tau, \mathbf{a}) = \sum_{\Delta,\lambda} m_{\Delta,\lambda}\chi_\lambda^F(\mathbf{a})\tau^\Delta, \quad H_H(\tau, \mathbf{1}) = \sum_\Delta \underbrace{\sum_\lambda m_{\Delta,\lambda}\dim\lambda}_{N_\Delta}\,\tau^\Delta. \tag{3.34}$$

Unrefined data retains graded dimensions N_Δ but cannot uniquely recover flavor representations. It is sufficient if only those counts or an unrefined PL are desired, subject to the same Higgs/HL and syzygy caveats. Highest-weight generating functions are a possible later representation-level summary [21].

Proposed implementation changes

Preserve flavor characters in the letter builder and under every Adams operation. Complex blocks use $R \otimes \mathbf{n}$ and $\overline{R} \otimes \overline{\mathbf{n}}$; real blocks use the $Sp(n)$ defining representation; pseudoreal blocks use the $SO(m)$ vector. Project only gauge factors, leaving flavor characters in the output. Simple-factor LiE/cache machinery can be reused, but the coefficient ring, parsers and serialization must support flavor labels and explicit abelian flavor charges. A refined PL must substitute *all* flavor variables. Share flavor-block construction in a lower-level helper to avoid a circular import between properties and index.

4 Package `common`

4.1 Flavor symmetry and exactly marginal gauge couplings

MODULE `common/n2_theory_properties.py`

The property layer reuses anomaly-validated matter and groups it into flavor blocks. The symmetry is the commutant of the gauge action on the hyper space, with its compatible reality structure. At zero masses the connected factors are

$$F_{\mathcal{R}} = \begin{cases} U(n), & \mathcal{R} \text{ complex, } n \text{ full hypers,} \\ Sp(n), & \mathcal{R} \text{ real, } n \text{ full hypers,} \\ SO(m), & \mathcal{R} \text{ pseudoreal, } m = 2n_{\text{full}} + n_{\text{half}}. \end{cases} \tag{4.1}$$

Here $Sp(n)$ has rank n and defining dimension $2n$, often written $USp(2n)$. Complex-conjugate labels are merged into a single block, including simultaneous conjugation of all product factors. A pseudoreal block counts half-hyper units. An $SO(1)$ block has no continuous flavor symmetry. Source: Hyper reality in [3]; flavor/commutant background in [17]; the explicit grouping rule is the project's implementation of that construction.

The helpers `_simple_flavor_blocks`, `_product_flavor_blocks`, `_canonical_complex_key` and `_calculate_flavor_symmetry` carry out this grouping. This is flavor-symmetry metadata only: it is not used to refine the current index. Disconnected components, faithful global quotients and strong-coupling accidental enhancements are not determined by this routine.

For an anomaly-free conformal Lagrangian theory with s simple factors, the code assigns one exactly marginal complex gauge coupling per factor:

$$\tau_a = \frac{\theta_a}{2\pi} + \frac{4\pi i}{g_a^2}, \qquad \dim_{\mathbb{C}} \mathcal{M}_{\text{conf}} = s. \tag{4.2}$$

This is the local dimension of the Lagrangian conformal manifold, not a description of its global S-duality quotient or singular loci. The τ_a in this equation is a coupling, distinct from the HL grading τ. Source: Lagrangian conformality [3]; $\mathcal{N} = 2$ supersymmetry-preserving marginal deformations in Córdova–Dumitrescu–Intriligator, §3.2.2/Table 22 [18].

Examples

$SU(3)$ with six full fundamental hypers has a $U(6)$ flavor block and one exactly marginal gauge coupling. $SU(2)$ with four full fundamentals is instead a pseudoreal block with eight half-hyper units, hence $SO(8)$ flavor. The distinction follows from total representation reality, not just multiplicity.

4.2 Central charges and property orchestration

For a conformal Lagrangian realization, define the effective numbers of vector and hypermultiplets by representation dimensions:

$$n_v = \sum_a \dim \mathfrak{g}_a, \qquad n_H = \sum_h n_h \epsilon_h \prod_a \dim R_{ha}. \tag{4.3}$$

The Weyl-anomaly coefficients are

$$a = \frac{5n_v + n_H}{24}, \qquad c = \frac{2n_v + n_H}{12}. \tag{4.4}$$

These include free gauge-singlet hypers. The code checks positivity and the equivalent identities $4(2a - c) = n_v$, $4(5c - 4a) = n_H$. Source: Shapere–Tachikawa, free-multiplet values in Eqs. (2.5)–(2.6) [15].

An additional mathematical cross-check with the Lagrangian Coulomb spectrum is

$$4(2a - c) = \sum_i (2d_i - 1) = \dim \mathfrak{g}. \tag{4.5}$$

The final equality follows from $\sum_i (d_i - 1) = |\Phi^+|$ and $\dim \mathfrak{g} = \ell + 2|\Phi^+|$. The more general Coulomb-dimension relation requires the assumptions discussed in [15], §5.1, Eq. (5.4); the project does not infer arbitrary non-Lagrangian central charges from it. The cross-check with degrees is explanatory here, not a separate implemented validation step. For $SU(3)$ with six full fundamentals, $n_v = 8, n_H = 18$, so $a = 29/12$, $c = 17/6$ and $4(2a - c) = 3 + 5 = 8$.

Basic properties without index calculation

`calculate_n2_theory_properties(data)` validates the input and returns flavor metadata, gauge marginal couplings, conformal-manifold dimension and exact central charges and disconnected-sector factor IDs. It never computes a full index, a Coulomb index or the Coulomb spectrum; those keys are omitted from the basic dictionary.

Basic property key	Representation
`group`	Gauge-group display name.
`lagrangian_scft_candidate`	Combined SCFT-candidate flag.
`flavor_symmetry`	Flavor-factor metadata.
`conformal_manifold_dimension`	Number of marginal gauge couplings.
`exactly_marginal_gauge_couplings`	Gauge-factor IDs.
`central_charges`	Exact rational a, c only.
`disconnected_sectors`	Materialized tuple of factor-ID tuples; JSON arrays on serialization.

For valid non-candidates, flavor metadata remains available; central charges and conformal-manifold dimension are `None`; sectors are also `None`, with no marginal gauge couplings. Malformed input raises an error. The basic calculator is descriptive; the database additionally rejects a theory that fails anomaly or conformality checks before inserting its theory data.

Separate index-related properties

The direct `index.n2_theory_index.calculate_index` API computes a formal gauge projection without imposing the SCFT-candidate gate. The following `common` wrappers enforce that gate before calculating index properties.

`calculate_n2_theory_indices(data,order=18,max_dimension=90)` validates once, then computes the full index, the gauge-degree Coulomb index and the complete spectrum. Full order N and Coulomb dimension D are independent, inclusive cutoffs. Omitting either argument uses the corresponding module default. Valid non-candidates receive `None` for all five fields.

Index-related key	Representation
`superconformal_index`	Serialized flat Laurent polynomial string.
`superconformal_index_order`	Nonnegative integer N; default 18.
`coulomb_branch_index`	Serialized x-series expression string.
`coulomb_branch_index_max_dimension`	Exact nonnegative rational D; default 90.
`coulomb_branch_spectrum`	Sorted tuple of exact `Fraction` dimensions, with multiplicities; independent of N, D.

The individual `calculate_superconformal_index(data,order=...)` and `calculate_coulomb_branch_index(data,max_dimension=...)` wrappers retain raw Sage results. `calculate_coulomb_branch_spectrum(data)` returns the complete tuple. The calculator does not write to MySQL or decide whether a stored result should be replaced; the database API applies that policy.

The property CLI defaults to basic properties. Select the second stage with `--indices`, optionally `--index-order` and `--coulomb-max-dimension`; supplying cutoffs without `--indices` is rejected. `calculate_n2_theory_properties_from_file` also stays basic.

4.3 Shared exact arithmetic, FORM execution and JSON

MODULE `common/number_utils.py; common/form_utils.py; common/json_utils.py`

These modules hold common utilities formerly duplicated in physics modules. Their role is to keep exact arithmetic and external-process handling consistent.

Exact-number contracts

Routine	Validation
`as_integer`	Rejects Python booleans/floats, including `1.0`; requires exact equality with the converted integer.
`as_nonnegative_int`	Calls `as_integer`, then requires $n \geq 0$.
`as_nonnegative_fraction`	Accepts supported exact rational objects or integer/fraction text; rejects Python booleans/floats and negative values.
`as_positive_fraction`	Adds the strict $q > 0$ condition.

Use `Fraction(6,5)` or Sage `QQ(6)/5` for a Coulomb dimension; use integer counts for Dynkin labels, hyper multiplicities and cutoffs in t. The rational-text format is an integer or fraction such as `"6/5"`, not decimal text such as `"1.2"`. Accepting integral floats would broaden the shared API everywhere, not just in the Coulomb module; the current behavior is deliberately strict. These validators are type/value checks, not physical unitarity or representation-existence proofs.

One FORM execution boundary

`run_form(program,form_executable=...,timeout=...,form_threads=1,tform_executable="tform")` writes a FORM program in an isolated temporary directory, invokes the executable with quiet output, and returns the output text. One thread selects FORM; larger counts select TFORM with `-wN` (Section 3.2.5). It turns missing executables, timeouts, nonzero exit status or reported stderr into errors with useful context. Both the full index and Coulomb PE/PL reuse this function; there is no need for separate subprocess wrappers in each module.

`split_top_level` and `split_signed_terms` support parsing expressions without splitting inside parentheses or mistaking an exponent sign for a new term. Physics-specific parsers remain with their own modules because character factors and Puiseux powers require different validation. Source: Execution and parser contracts from the source; FORM program syntax in [12].

Serialization is exact, but precision metadata is separate

The JSON helpers encode a fraction as

```
{"numerator": 6, "denominator": 5}
```

Tuples become arrays, dataclass records become structured objects, and the separate index-property layer serializes Sage indices as expression strings. For example, the spectrum $(2, 3)$ is stored as an array of the two exact fractions, not as the polynomial $x^2 + x^3$.

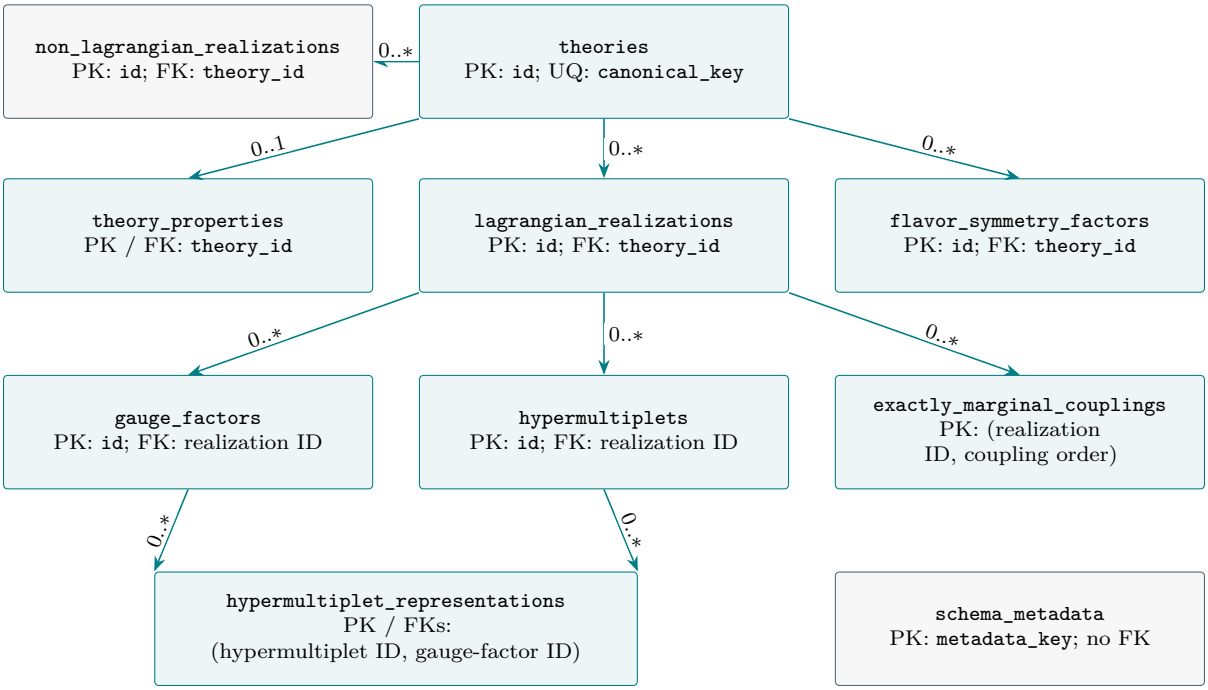
Index strings do not embed their trusted truncation order. The separate calculator returns it explicitly, and MySQL schema 1 stores the two cutoffs beside the expressions and in combined JSON. The last nonzero term is not a substitute for this metadata. Future non-Lagrangian imports still need their own validated input/provenance workflow; that importer is not implemented.

4.4 MySQL persistence and schema version 1

MODULE `common/n2_theory_db.py`

The database distinguishes a theory, its shared properties, and its realizations. Flavor factors belong to the theory; gauge factors, hypermultiplets and exactly marginal couplings belong to a Lagrangian realization. The non-Lagrangian table is a placeholder, with no implemented import workflow.

Database relationships



Arrows run from parent to child; labels give the number of children per parent allowed by the schema. Each child has exactly one parent along each foreign key. PK = primary key; FK = foreign key; UQ = unique key. "Realization ID" abbreviates `lagrangian_realization_id`. All declared FKs use `ON DELETE CASCADE`.

How product-group matter is represented

One `hypermultiplets` row holds a complete tensor-product representation and its submitted `kind` and `multiplicity`. Its rows in `hypermultiplet_representations` identify the representation under each gauge factor, including singlets. The latter table joins matter to gauge factors; it does not create a separate hypermultiplet for each factor.

> **Shared flavor versus the submitted matter split**
>
> Production schema 1 retains `half_hyper_units` in the shared flavor table and JSON, with $m = 2n_{\text{full}} + n_{\text{half}}$. The original split remains in realization rows and input/anomaly JSON. Public property-calculator output still reports both counts; the database strips them from its shared-property copy.

4.5 Database table catalog and keys

All tables use InnoDB and `utf8mb4`. IDs are unsigned `BIGINT`s; standalone `id` keys auto-increment. Exact rational values use JSON numerator/denominator records or separate numeric columns, as detailed below.

Table	Keys and stored data
`schema_metadata`	PK `metadata_key`; `metadata_value` holds the schema version (current value `"1"`). Independent of theory rows.
`theories`	PK `id`; UQ `canonical_key`. Display name and creation/update timestamps. A new Lagrangian theory uses `lagrangian:<hash>` as its key.
`theory_properties`	PK/FK `theory_id` → `theories.id`. Flavor group/rank/dimension, conformal-manifold dimension, exact central-charge JSON, separate full-index, Coulomb-index and Coulomb-spectrum JSON, two cutoff columns, sector count/factor-ID JSON, and combined `properties_json`. Generated, indexed `DECIMAL(65,30)` columns approximate a, c for queries.
`flavor_symmetry_factors`	PK `id`; FK `theory_id` → `theories.id`; UQ (`theory_id`, `factor_order`). Group, Lie algebra/rank/dimension, representation reality, `half_hyper_units`, and `gauge_representation_json`. The full/half split is stored per realization.
`lagrangian_realizations`	PK `id`; FK `theory_id` → `theories.id`; UQ `canonical_hash` (64-character SHA-256). Gauge group/count, anomaly and conformality flags, input/anomaly/coupling JSON, creation timestamp.
`non_lagrangian_realizations`	PK `id`; FK `theory_id` → `theories.id`. Construction type, optional source reference, `data_json`, creation timestamp. Reserved.
`gauge_factors`	PK `id`; FK realization ID → `lagrangian_realizations.id`. Separate UQs on (realization ID, `factor_order`) and (realization ID, `factor_key`). Group, Cartan type/rank, exact vector, matter and b_0 fractions, beta/global-anomaly flags and Witten parity.
`hypermultiplets`	PK `id`; FK realization ID → `lagrangian_realizations.id`; UQ (realization ID, `hyper_order`). Name, `kind` (full/half), submitted multiplicity, total tensor-product dimension and reality.
`hypermultiplet_representations`	Composite PK (`hypermultiplet_id`, `gauge_factor_id`); separate FKs to `hypermultiplets.id` and `gauge_factors.id`. Name, Dynkin-label JSON, dimension/reality, exact Dynkin-index and beta-contribution numerators/denominators.
`exactly_marginal_couplings`	Composite PK (realization ID, `coupling_order`); FK realization ID → `lagrangian_realizations.id`. Stores `gauge_factor_key` as text; that key has no additional FK to `gauge_factors`.

Serialization and integrity boundaries

Full and Coulomb indices are JSON expression strings. Coulomb spectra are arrays of exact fractions; central charges are exact a, c fraction records. The separate computed fields and their cutoffs are mirrored into `properties_json` when filled. Initial imports leave computed columns NULL and omit those keys from the basic JSON. Gauge beta fractions and representation invariants use `DECIMAL(65,0)` integer columns. The writer keeps both ends of each matter–gauge relation within the same realization; the two individual FKs do not themselves enforce that match.

4.6 Canonical matter and production import behavior

The canonical Lagrangian hash is SHA-256 of a normalized payload. Names resolve to Dynkin labels, repeated matter entries combine, and zero multiplicities vanish. Conjugate choices use the lexicographically larger label tuple, favoring fundamentals. Product representations identify only simultaneous conjugation of all factors. Matter order is ignored; gauge-factor order remains significant.

Full/half equivalence for pseudoreal representations

For each complete pseudoreal gauge representation, aggregate before pairing:

$$m = 2n_{\text{full}} + n_{\text{half}}, \qquad (n_{\text{full}}^{\text{canonical}}, n_{\text{half}}^{\text{canonical}}) = (\lfloor m/2 \rfloor,\ m \bmod 2).$$

Four full $SU(2)$ fundamentals, eight halves, or two full plus four halves now have the same hash. An odd remaining half stays in its own irrep. For products, the rule uses the reality of the entire tensor product. Real/complex matter remains full; invalid half-hyper input is rejected by the checker.

Production schema 1

The test period has ended. `SCHEMA_VERSION=1` names the complete current ten-table layout, including decimal charge indexes, separate index/spectrum fields, both cutoffs and disconnected-sector metadata. It is not the early test-period schema once numbered 1.

`initialize_database` executes the current `CREATETABLEIFNOTEXISTS` definitions and records version 1 when the metadata row is absent. Existing version 1 is accepted; any other recorded version is rejected. Initialization does not alter existing table definitions, relabel an old database or enrich its rows. Pre-release migrations, their dispatch loop, the sector backfill API and the migration CLI were removed with their tests. There is no longer a universal update command.

An existing test database must not be assumed compatible from its version number alone. No user database was relabelled or modified by this cleanup. SQLite caches have their own independent schema version.

`_insert_shared_properties` compares only basic physical properties, ignoring the obsolete flavor split in legacy JSON. Matching JSON is normalized without erasing existing index data. Spectrum filling now belongs to the separate worker. Physical mismatches still reject attachment; shared flavor metadata includes gauge labels and factor IDs, so this is not a general comparison of arbitrary dual realizations.

`store_lagrangian_theory` checks anomalies/conformality and calculates basic properties before looking up the hash. It performs no index or spectrum calculation. A duplicate returns the existing IDs with `inserted=False` and does not change the name, `updated_at`, realization or shared data. A new import uses parameterized SQL and one explicit data transaction. Schema initialization precedes that transaction. There is no automatic recognition of dual descriptions; attachment with `theory_id` still requires matching basic shared properties.

4.7 Independent precision and replacement rules

The database stores one set of physical properties per theory. A Lagrangian realization supplies the input for later calculation; different realizations attached to the same theory share its indices and Coulomb spectrum. Initial imports run the anomaly/conformality checks and calculate basic properties only. Both indices, the spectrum and both precision columns start as SQL `NULL`; their keys are initially absent from `properties_json`.

The separate calculator returns full-index precision $N \in \mathbb{Z}_{\geq 0}$ and Coulomb precision $D \in \mathbb{Q}_{\geq 0}$. These are inclusive cutoffs:

$$\mathcal{I}_{\leq N} = \sum_{a=0}^{N} t^a P_a(y,u), \qquad \mathcal{I}_{\mathrm{C},\leq D} = \sum_{0\leq\Delta\leq D} c_\Delta x^\Delta.$$

A zero boundary coefficient does not reduce the trusted cutoff. For example, $1 + x^2 + x^4$ computed through $D = 5$ must retain precision five, even though its largest nonzero exponent is four. The complete gauge-invariant generator spectrum is independent of either cutoff; it is not truncated with the index.

For each of the two indices separately, let V be its stored value and $p_{\mathrm{old}}, p_{\mathrm{new}}$ its stored and submitted cutoffs. The write rule is

$$\text{replace or fill} \iff V \text{ is missing} \quad \text{or} \quad \big(p_{\mathrm{old}} \text{ is known and } p_{\mathrm{new}} > p_{\mathrm{old}}\big).$$

Stored / submitted state	Database action
Stored index missing	Fill it and record the supplied cutoff.
Known, strictly higher cutoff	Replace that index and its cutoff.
Equal or lower cutoff	Keep the existing string and precision unchanged.
Legacy index, unknown cutoff	Keep it; report `unknown_precision`.
Omitted or `None` component	Leave the corresponding stored data alone.
Supplied index without a cutoff	Reject the payload before opening the write transaction.

For example, an update from $(N, D) = (18, 90)$ to $(24, 60)$ upgrades the full index only. The Coulomb index remains at dimension 90. Equal-order submissions do not replace even a different string; this API is not a correction/force-update mechanism. The caller supplies the trusted calculation cutoff; the writer does not infer or prove precision from the expression's last monomial.

A missing spectrum is filled. An identical complete spectrum is retained, with multiplicities preserved; a conflicting spectrum raises an error and rolls back that update call. This policy differs from comparison of truncated indices.

Implementation source: `common/n2_theory_properties.py`, `common/n2_theory_db.py`; the inequalities above express the implemented cutoff contract, not an additional physical conjecture.

4.8 Production precision metadata and atomic writes

Schema 1 directly includes the full-index and Coulomb precision columns in `theory_properties`. Their role is independent of schema versioning.

Column / result key	Stored representation
`superconformal_index_order`	`BIGINT UNSIGNED`; inclusive integer N.
`coulomb_branch_index_max_dimension_json`	Exact numerator/denominator JSON for D.
`coulomb_branch_index_max_dimension`	Corresponding key in calculator output and combined JSON.

The three computed values still use `superconformal_index_json`, `coulomb_branch_index_json` and `coulomb_branch_spectrum_json`. Successful updates mirror changed values and cutoffs into `properties_json` while preserving basic properties and other computed components.

No schema migration or automatic data backfill is implemented. Fresh-schema DDL precedes the explicit data transaction and is not rolled back with it.

When an old result's original requested cutoff is independently known, record it before normal upgrades:

```
from common.n2_theory_db import record_lagrangian_index_cutoffs

# Use these numbers only if they are the verified original cutoffs.
record_lagrangian_index_cutoffs(
    connection, realization_id, order=18, max_dimension=90,
)
```

This function fills unknown precision only. It rejects changing an already known cutoff or annotating an index that is absent. It changes neither index expression. Merely seeing a last term of degree 18 does not establish that the original calculation was requested through 18.

`update_lagrangian_indices(connection,realization_id,indices)` accepts `calculate_n2_theory_indices`'s output or a subset of computed components with their matching cutoffs. It opens a short transaction, locks the parent theory before the shared property/realization rows with `SELECT ... FOR UPDATE`, rechecks the current cutoffs, and merges accepted changes into the columns and JSON together. Failure rolls back the call; a no-op does not change the theory's `updated_at` timestamp.

Calculations run before this lock is acquired. Thus a slower low-order worker cannot overwrite a higher-order result committed while it was calculating. Use an initialized connection with no caller-owned transaction for update and legacy-cutoff APIs. These APIs manage their own commit/rollback boundary.

4.9 Disconnected-sector properties and persisted factor IDs

The public `split_disconnected_sectors` partitions the gauge/matter graph as described in Section 3.4. The basic property calculator returns a materialized tuple of tuples of input factor IDs:

```
"disconnected_sectors": (("left", "middle"), ("right",), ())
```

The example has two interacting components and one combined free-hyper sector. IDs are input keys, not Cartan types. Zero-multiplicity matter does not connect components; all free hypers are assigned once to the final empty-factor sector. A single simple input normally uses factor ID `gauge`. The property is `None` for a non-SCFT candidate and does not require index or Coulomb-spectrum calculation.

Stored field	Value for the example
`disconnected_sector_count`	`3`
`disconnected_sectors_json`	`[["left","middle"],["right"],[]]`
`properties_json` keys	`disconnected_sector_count` and `disconnected_sectors`

The dedicated columns are nullable `INTUNSIGNED` and `JSON`. Count is the number of returned sectors, including the optional combined free sector. NULL denotes unknown/unpopulated data.

Factor IDs belong to realizations. The stored description therefore uses the first stored realization (smallest `lagrangian_realizations.id`) for that theory. Other realizations and duplicate aliases preserve it; the sector IDs are excluded from shared physical-property equality checks. This prevents equivalent input names from making the theory-level sector record ambiguous.

A successful new import inserts the realization, then `_store_disconnected_sectors` writes both dedicated columns and their combined-JSON keys inside the import transaction. The helper only fills missing metadata for that first realization, preserving existing values and computed indices/cutoffs. Failures roll back with the import. Normal duplicate imports do not serve as a bulk enrichment operation.

The earlier sector schema-upgrade/backfill implementation was removed when the full layout became production schema 1. No initializer now scans saved inputs to populate old rows, and the removed migration CLI must not be used. `test_n2_theory_db_sectors.py` retains normal storage tests for connected, disconnected and free sectors, plus first-realization and duplicate-name behavior.

4.10 Separate worker, partial retries and reproducible use

The program `common/n2_theory_db_indices.py` streams stored Lagrangian inputs in bounded pages, using the smallest realization ID for each theory. It visits each shared theory once rather than recalculating the same shared properties for every attached realization.

1. Default mode selects any missing full index, Coulomb index or spectrum. SQL `NULL` and JSON `null` are both treated as missing.
2. `--upgrade` additionally selects indices with known lower cutoffs. Already sufficient components are not calculated. Complete legacy records with unknown precision are reported without recalculating those indices.
3. Each requested component is calculated outside a database transaction, then submitted to the locked update API. The stored state is checked again at write time, even if it differed when the job was selected.
4. Each successful component commits independently. A failure is reported for that component; subsequent components and theories continue. Re-running retries remaining missing or lower-order components and keeps earlier successes.

Multiple workers can duplicate a calculation; there is no persistent job-claim queue. The row-lock and cutoff policy protects data against downgrades, not against duplicated CPU work. A conflicting spectrum rolls back its own update call, not components already saved by earlier worker calls.

Run from the repository root with Sage's Python. Database credentials follow the existing `N2_DB_HOST`, `N2_DB_USER`, `N2_DB_PASSWORD` and `N2_DB_UNIX_SOCKET` environment conventions; port is a CLI option.

```
# Stage 1: validate and store basic properties.
sage -python -m common.n2_theory_db \
  anomalies/example_e6.json DATABASE --user USER
# Stage 2: fill missing components, defaults N=18 and D=90.
sage -python -m common.n2_theory_db_indices DATABASE --user USER
# Later: upgrade only components with lower known precision.
sage -python -m common.n2_theory_db_indices DATABASE --user USER \
  --upgrade --index-order 24 --coulomb-max-dimension 100 --limit 10
```

Options include `--host`, `--port`, `--unix-socket`, `--connect-timeout`, `--char-cache-database`, `--lie-executable`, `--form-executable`, `--timeout` and `--processes`. `--limit` bounds visited theory jobs, including reported legacy skips. The API `fill_lagrangian_indices` yields the same per-theory outcomes. Each outcome reports `updated_fields`, `skipped_fields` and `errors`; a final CLI summary counts processed and failed jobs. Exit codes are 0 for no component failures, 1 for component failures, and 2 for an outer configuration or database-level error.

Validation recorded during implementation, 12 September 2026. All 238 backend tests passed in 21.920 seconds, including an isolated MySQL 8.0.46 server, schema migration, concurrent upgrades, rollback, partial retries, JSON-null selection and real Sage/FORM/LiE calculations. This PDF update checks source contracts and document rendering; it does not rerun that suite or alter the user's database. Older cache-performance measurements remain dated evidence.

5 Package gui: anomaly, index, search and Settings

5.1 Editable input, saved settings and portable launch

The PyQt6 window loads `gui/n2_db.ui` and `gui/settings.ui` directly. The anomaly tab has two side-by-side areas: editable multiline gauge input and **Load theories...** on the left; a character-cache checkbox, read-only progress log and **Build/Stop** controls on the right. The anomaly, index and search tabs have separate controllers; index work, search/CSV export and deletion are described in Sections 5.6, 5.10 and 5.9.

Enter one gauge-group specification per nonempty line, separating factors by commas. These are instructions to enumerate matter contents, not complete Lagrangian JSON inputs. For example:

```
A1
A1, C2
G2
```

The loader appends one or more UTF-8 files in selection order, accepting a BOM and Windows line endings. It preserves existing text, inserts line boundaries, and makes the complete load one undoable edit. Cancel and empty files make no change. If any file cannot be read or decoded, the entire load is rejected with the filename and reason; the previous input remains editable and unchanged.

Settings / Preferences persists the following new fields alongside the existing database, cache and executable settings:

Setting	Default	Contract / saved key
Full-index maximum order	18	Inclusive nonnegative integer; `index/full_max_order`, matching `INDEX_MAX_ORDER`.
Coulomb maximum dimension	90	Inclusive nonnegative exact rational; `index/coulomb_max_dimension`, matching `C_INDEX_MAX_ORDER`. Fractions such as `6/5` are reduced and stored as strings.
CPU cores	−1	`tools/processes`; all logical CPUs via `os.cpu_count()`, or one if unavailable. Positive values bound workers; 1 is serial and 0 is invalid.
TFORM threads	1	`tools/form_threads`; exact positive integer. One selects FORM; above one selects TFORM.
TFORM executable	`tform`	`tools/tform_executable`; PATH name or full path.

Older settings files receive these defaults until saved. OK saves immediately; Cancel discards edits. A build uses a fixed settings/input snapshot. Existing QSettings persistence and native keyring credential handling are retained. Index workers divide the CPU budget by the TFORM count; anomaly workers do not. Existing preferences default to serial FORM until changed.

Activate any Sage environment providing the GUI requirements, then run from the project root:

```
sage -python -m pip install -r gui/requirements.txt
sage -python -B gui/n2_db.py
```

The command contains no machine-specific installation path. Python's optional `-B` suppresses bytecode writes (`.pyc`/`__pycache__`); it does not change the calculation or disable the project's SQLite caches.

5.2 Enumeration, SCFT checks and character-cache preparation

`gui/theory_builder.py` runs in a separate Sage process managed by the GUI. Credentials and settings arrive through stdin; the worker emits structured JSON log messages. The window remains responsive. The coordinator validates Cartan types, reports bad factors with their input line numbers, and initializes the MySQL schema before dispatching any candidate workers.

For each usable line, `enumerate_simple_theory_candidates` handles one factor and `enumerate_product_theory_candidates` handles multiple factors. Both come from `common.n2_theory_iter`; enumeration runs once in the coordinator, using the existing backend defaults. Its finite search omits free all-singlet matter. Results are subsequently checked by `calculate_n2_theory_properties` in `common.n2_theory_properties`. The candidate flag includes anomaly and conformality checks; enumeration alone does not establish that flag. Invalid candidates include their rejection reason. Unexpected checking failures are counted separately.

Valid candidates go through `store_lagrangian_theory` in `common.n2_theory_db`. It revalidates and atomically saves basic properties and a realization. New rows still have no computed indices or Coulomb spectrum. The log reports candidate totals, valid/invalid counts, additions, existing records, checking errors and database errors per group and overall. A valid candidate with a failed insert remains valid and also increments database errors; it is not relabeled physically invalid. Duplicate input lines run again and normally find the earlier records.

If **Build character decomposition cache** is enabled, collect the union of distinct pairs of Cartan type and Dynkin labels across all valid candidates, including those already stored or whose insertion failed. This includes every vector adjoint, matter factor, factor singlet and the conjugate orientation for a full hyper. After all gauge groups finish, call `index.char_decomposition_cache.build_decomposition_cache` for each pair with maximum weighted Adams order, where N is `index/full_max_order`:

$$K = \left\lfloor \frac{N}{2} \right\rfloor .$$

Thus the default $N = 18$ gives $K = 9$. A zero bound skips prebuilding. Existing decompositions are reused; each completed order reports total, computed and reused products. The cutoff follows the full-index letter grading explained in the cache chapter and is not a Hall–Littlewood cutoff rule.

This action uses the saved character-cache file, LiE executable, invocation timeout and CPU count. It does not calculate an index: the Coulomb cutoff, FORM executable and FORM-cache path are saved for index calculations but are not consumed by the anomaly build. Candidate pools finish before cache pools start, so their configured process counts do not multiply.

The existing group naming convention remains algebra-based: B_r is displayed as $\mathrm{Spin}(2r+1)$ and D_r as $\mathrm{Spin}(2r)$. No global-form migration was made in this session. Quotient groups, allowed line operators and discrete theta data are not stored by this implementation; changing the displayed name alone would not add them.

5.3 Parallel ownership, cancellation and durable logs

`gui/candidate_workers.py` uses spawned processes. With C candidates and a resolved CPU limit P, a group's pool has $\min(P, C)$ workers and at most twice that many submitted tasks. Each task contains one candidate. Available workers receive subsequent tasks dynamically; groups are completed in input order. Enumeration itself is serial and materializes the candidate list.

Each worker owns its Sage state and a lazily opened MySQL connection, reuses that connection and closes it during normal pool shutdown. Connections and cursors are never shared across threads or processes. Worker imports use `initialize_schema=False` after serial schema setup. Only the coordinator combines counters, representation sets and logs. A failed database connection is closed so later tasks can open another; an uncertain write is not replayed.

Stop stops new submissions, cancels queued work where possible, and signals workers before subsequent checks or imports. Already active calls finish and their results are drained before final totals are reported, retaining counts for committed inserts. Enumeration must finish before it observes Stop; cache generation stops after the current order commits. Closing the window requests the same cooperative stop. Previously committed rows remain available. A crashed pool is reported, including potentially unknown database outcomes; remaining candidates are not silently retried after the crash.

Every Build attempt creates a separate UTF-8 file under `logs/` at the project root, including attempts that fail validation before work starts:

```
logs/log_anomalies_YYYYMMDD_HHMMSS_ffffff.log
[2026-09-14 14:00:43.455+09:00] [INFO] message
```

The filename uses local time with microseconds and exclusive creation, so an existing file is not overwritten. Both GUI and file show local timestamps with milliseconds and UTC offset, plus `INFO`, `WARNING` or `ERROR`. Each line of a multiline message receives the same prefix. The GUI retains timestamps supplied by the build process; candidate-pool results are timestamped when the coordinator emits them. GUI-origin messages use their logging time.

Progress is INFO, rejected candidates are WARNING, and failures are ERROR. Unstructured stdout is INFO and stderr is WARNING; abnormal process exits are also reported as ERROR. Passwords are redacted before display and file output. Messages are flushed as they arrive, and logs close on completion, failure or Stop. File errors are shown in the GUI, which continues logging on screen. Generated logs are ignored by Git.

Implementation sources: `gui/n2_db.py`, `gui/anomaly_tab.py`, `gui/logging_utils.py`, `gui/theory_builder.py`, `gui/candidate_workers.py`, the editable UI files, and `gui/README.md`. The configured worker limit also applies to character-cache generation; it is not a guarantee of proportional speedup.

5.4 Concurrent imports: deadlock fix and bounded retries

Each theory import is one explicit InnoDB transaction. Equivalent concurrent imports are resolved by existing unique keys: a losing duplicate-key transaction rolls back and returns the committed winner with `inserted=False`. That 14 September fix changed neither the then-current schema nor server isolation.

The former fresh-import path selected a properties row before inserting it:

```
SELECT properties_json FROM theory_properties
WHERE theory_id = %s FOR UPDATE
```

The parent theory was newly inserted, so the properties row could not yet exist. Under REPEATABLE READ, the absent-row lookup could lock an index gap. Independent transactions could hold compatible gap locks, then each wait for the other's lock when inserting. The local MySQL deadlock report identified this cycle in the `theory_properties` primary index. See the MySQL 8.0 manual, InnoDB Locking: gap and insert-intention locks (checked 14 September 2026) for the lock semantics.

The fix passes `new_theory=True` to `_insert_shared_properties` only when this transaction inserted a new parent. It inserts the properties directly and skips the missing-row locking read. Attaching a realization to an existing theory keeps the locking read for comparison and enrichment. Atomic rollback, duplicate detection and existing index preservation remain in place.

For a residual deadlock (1213) or lock timeout (1205), the importer explicitly rolls back and retries the whole transaction at most twice: three total attempts, with 50 ms and 100 ms waits. Exhaustion propagates to the candidate log and database-error count. Other failures, including uncertain connection/commit outcomes, propagate without automatic replay. A rollback failure is reported with the original error. A later Build can recover missing imports while recognizing already committed realizations.

Controlled check, 14 September 2026	Before	After
256 distinct valid candidates, 8 workers: failed inserts	93	0
Same run: successfully inserted theories	163	256
Synchronized pair: transaction rollbacks	1	0

The fixture used A_1 with four full fundamentals and 1–256 free full singlets to produce distinct valid inputs without increasing gauge-algebra complexity. It exercises checking/storage directly; the standard enumerator excludes these free singlets. The fixed pair and stress runs added zero InnoDB deadlocks. These are local observations on a disposable MySQL 8.0.46 instance, not a promise that every workload is deadlock-free. Regression tests retain both the synchronized pair and the 256-candidate case.

5.5 Test-harness repairs and validation evidence

The session also corrected test assumptions exposed by password-protected MySQL and unrelated active clients. The index-worker CLI test now forwards the configured test password and Unix socket through the application's `N2_DB_PASSWORD` and `N2_DB_UNIX_SOCKET` conventions, restoring the environment afterward and including stderr in failures. Parallel cleanup checks track only connections opened by that test, including spawned workers, and wait for those IDs to disappear. Unrelated connections are left alone. A negative regression confirms that an owned connection leak is still detected.

Validation	Result and scope
Backend, after deadlock fix	268 tests passed in 33.518 s, no skips; actual Sage/FORM/LiE and password-protected disposable MySQL 8.0.46.
Focused concurrency/unit run	37 tests passed, including the synchronized pair, 256 candidates and existing database unit regressions.
GUI, documentation refresh	32 offscreen tests passed in 0.869 s; temporary settings and credential test doubles, not the personal desktop vault.

These two results were recorded during the 14 September implementation and documentation refresh. Later combined-suite evidence follows in this reference. Earlier 238-test database and 209-test cache results remain historical snapshots. No production database was used for this validation, and the isolated server was shut down afterward.

Reproduce with Sage and an explicitly disposable test database. Set `N2_TEST_MYSQL_DATABASE`, `N2_TEST_MYSQL_HOST`, `N2_TEST_MYSQL_PORT`, `N2_TEST_MYSQL_USER`, `N2_TEST_MYSQL_PASSWORD`, and optionally `N2_TEST_MYSQL_UNIX_SOCKET` for that instance before running:

```
PYTHONDONTWRITEBYTECODE=1 sage -python -B \
  -m unittest discover -s test -p 'test_*.py' -v
QT_QPA_PLATFORM=offscreen PYTHONDONTWRITEBYTECODE=1 \
  sage -python -B -m unittest test.test_gui_n2_db \
  test.test_gui_index_tab test.test_gui_logging_utils -v
```

Live database tests reset their configured test tables. Omitting `N2_TEST_MYSQL_DATABASE` skips those integrations. The repository's `all_test.sh` currently supplies local connection values; this session fixed the test harness, not that script's configuration.

Regression sources: `test/test_gui_n2_db.py`, `test/test_gui_theory_builder.py`, `test/test_gui_candidate_workers.py`, `test/test_n2_theory_db.py`, and the CLI environment regression in `test/test_n2_theory_db_indices.py`.

5.6 Tab controllers, reusable logging and retained index searches

`gui/n2_db.py` owns the main window, settings and coordination between tabs. `AnomalyTabController` in `gui/anomaly_tab.py` owns anomaly input loading, Build/Stop, process output and build state. The parallel `IndexTabController` in `gui/index_tab.py` owns index search, selection, calculation and retained results. Both are `QObjects`; their action methods are connected to widgets loaded from `gui/n2_db.ui`.

The index tab has two side-by-side areas. The left contains **Search missing or lower-order indices**, **Select all**, and checkboxes grouped by the theory's ordered gauge-group factors, with counts. The right contains a read-only log and **Calculate index**, which becomes **Stop** during calculation. Select all checks or clears every listed group; individual selections update its state too.

`gui/index_search.py` runs as a separate Sage process. It uses the saved MySQL settings and `iter_lagrangian_index_jobs(upgrade=True)`, without schema initialization or writes. It passes the saved full-index order N and exact Coulomb cutoff D. A job qualifies when a full index, Coulomb index or spectrum is SQL NULL/JSON null, or when a recorded full/Coulomb cutoff is strictly below its own requested cutoff. The two comparisons are independent; a high Coulomb cutoff does not satisfy a low full-index order. Equal/higher known cutoffs need no upgrade. Unknown precision alone does not qualify; a missing other component can still select that theory. One Lagrangian realization is retrieved for each shared theory.

Successful searches retain complete theory/realization IDs, factor and hyper inputs, needed-field metadata and source identity in the controller. `retrieved_jobs` returns all jobs and `selected_jobs` returns checked groups' jobs as copies. The next calculation uses this snapshot without a second database-wide search. New groups begin unchecked; a successful refresh preserves checks for still-present groups. An empty successful search clears the list; a failed/incomplete search retains the last successful snapshot. Changing the source database clears it; changing a cutoff does not. The snapshot is in memory only and does not survive restarting the application.

Shared log implementation. `gui/logging_utils.py` provides `redact_message`, `make_log_record`, `format_log_message`, and `GuiLogger`. A logger accepts an append callback, secrets to redact, and an optional file started with a configurable prefix. Both tab controllers use this API, so future logs can reuse the same timestamp, level, multiline, redaction and flushing behavior. Each index search and calculation attempt creates its own file, including failures before worker startup:

```
logs/log_index_YYYYMMDD_HHMMSS_ffffff.log
```

The file and read-only widget receive the same redacted messages. A file error is reported on screen without stopping the work. The anomaly prefix remains `log_anomalies`. Files close after completion, failure or cancellation.

5.7 Multiprocessing index calculation and partial progress

`gui/index_calculator.py` is the Sage coordinator started by **Calculate index**. It distributes the selected retained theories across spawned processes. With C selected theories, resolved CPU budget P and TFORM count T, at most $\min(C, \max(1, \lfloor P/T \rfloor))$ workers run, with at most twice that many outstanding tasks. One task contains one theory; scheduling is dynamic. The setting -1 resolves to all logical CPUs, with fallback one. Section 3.2.5 explains engine selection and memory limits.

Each worker owns its MySQL connection, opened against the initialized schema, and closes it at normal shutdown. Connections never cross process boundaries. The worker calls `common.n2_theory_db_indices.calculate_index_job` with saved full/Coulomb cutoffs, executable paths, timeout and both cache filenames. Inner LiE/character work uses one process; FORM receives the selected thread count. The worker passes its own theory connection to the disconnected-sector reuse path described in Section 3.4.

1. Recheck each retained theory/realization using `needed_lagrangian_index_fields` with the current saved N, D. This targeted check can add newly needed components after settings change or omit work already completed by another writer. It does not repeat the database-wide search or expand the retained theory selection.
2. Compute missing or lower-order components outside write transactions. A stored full index does not constrain the separate Coulomb cutoff. The complete spectrum is filled only when absent.
3. Save each successful component independently with upgrades enabled (`missing_only=False`). Locked comparisons preserve concurrent equal/higher results and unknown precision; a failure in another component does not erase successful commits.
4. Publish outcomes. Confirmed complete jobs are removed from the retained selection when the run ends. Failed, stopped or unconfirmed jobs remain for retry; a later retry rechecks which components still need work.

A manager-backed queue streams log records from workers to the coordinator. The coordinator drains results and logs while tasks are active, avoiding the feeder-thread shutdown pattern in which a child waits on a full, undrained ordinary multiprocessing queue. Scheduling remains bounded even for a long selection. Unexpected worker exit is reported rather than silently retried.

Stop and close. Stop cancels queued work where possible and lets active components finish and save before workers shut down. Window close requests the same cooperative stop. Search, anomaly Build and Preferences are mutually blocked during a calculation. A read-only search can be cancelled on close. These ownership and shutdown rules address process/queue deadlocks; they do not eliminate mathematical workloads that take a long time.

5.8 Database lock ordering and the test-permission repair

The index writer and legacy-cutoff recorder lock the parent theory before property/realization rows. Existing-theory anomaly attachment uses the same parent-first order. This removes the earlier child/parent cycle between imports and index updates. The fresh-import missing-row gap-lock fix remains: a newly inserted theory gets a direct property insert rather than a locking lookup for an absent property row.

For MySQL errors 1205 (lock wait timeout) and 1213 (deadlock), the index write and cutoff APIs roll back the whole short transaction, wait 50 then 100 ms, and retry at most twice. Each attempt reads the current state again and merges only permitted fields; the expensive index is not recomputed. Connection failures and failed rollbacks are not replayed. Other clients can still introduce contention, so the result is bounded recovery, not a claim that all possible database deadlocks are impossible.

The 14 September recorded index run passed 338 tests in 55.104 s, including the real GUI search/calculation protocol, custom caches, partial failures, cooperative Stop, abrupt worker exit, connection cleanup, an eight-worker 256-theory stress case and targeted parent-row contention. The controlled stress and contention runs added no observed deadlocks.

The later `all_test.sh` failure was a test-account permission problem: concurrency tests queried server-wide `INNODB_METRICS`, which requires privileges the database-scoped test account does not have. The repaired tests observe rollback calls on their own connections, including successfully retried lock failures. Cleanup inspects only connections owned by that test user. No global PROCESS or SUPER grant is required. The recorded post-repair suite passed 339 tests. Direct-file/module cache-builder resume coverage remains in `test/test_build_decomposition_cache.py`; its appearance at the start of the submitted test log did not identify it as the failing test.

Test organization. All new tests and test helpers live in `test/`. The moved GUI tests use `test_gui_` names; the opt-in real credential-vault check is `test/check_secret_service.py`. Application files under `gui/` contain runtime code only. `all_test.sh` discovers the combined suite from `test/`. Live integration fixtures reset only the configured test database and need database-scoped privileges; ordinary GUI tests use temporary settings and fixture credentials.

5.9 Password-confirmed deletion from Settings

The MySQL section of Settings has **Delete all database contents...**. `SettingsDialog` snapshots the displayed database, host, port, user and timeout, including unsaved edits, plus the effective Unix socket when set. `DeleteDatabaseDialog` shows that target, an irreversible-deletion warning, a fresh masked password field, and Delete/Cancel. The saved password is not prefilled. Opening or cancelling the dialog performs no database operation.

The editable layout is `gui/database_clear_dialog.ui`. Python loads it with `uic.loadUi`, as it loads the main and Settings forms. The UI file owns labels, layout, password echo mode, disabled initial Delete button, default Cancel button and tab order. `gui/database_clear_dialog.py` supplies the target text and platform warning icon, connects signals, and controls the asynchronous operation. It no longer constructs the layout.

1. A nonempty freshly entered password enables Delete. Password whitespace is preserved. Delete starts a separate Sage process for `gui.database_clear`; target, password and explicit confirmation travel through stdin, never through command-line arguments or saved preferences.
2. The worker authenticates a fresh MySQL connection using the entered password and `initialize_schema=False`. This is database authentication, not comparison with the previously saved credential.
3. `delete_all_theory_data` checks production schema version 1, starts a transaction, executes `DELETE FROM theories`, and commits. Foreign-key cascades remove the associated property, realization, gauge, hyper, flavor, index and spectrum data. Tables, schema metadata, account and SQLite caches remain. There is no DROP, TRUNCATE or auto-increment reset. Exceptions trigger rollback.
4. A confirmed result returns the deleted parent-theory count and clears the index tab's retained jobs, even if Settings is subsequently cancelled.

The dialog remains responsive but disables password input and cancellation during deletion. Dialog, Settings and main-window close wait for the worker. Settings is unavailable during a build, search or calculation. Wrong-password rejection allows a fresh entry; errors and logs redact secrets. A lost worker or connection can make the commit outcome uncertain, so the UI reports an unconfirmed result and advises inspecting the database before retrying. The helper guarantees the transactional attempt, not knowledge of a commit after a lost connection.

Recorded validation, 16 September. The full suite passed 370 tests in 65.584 s with no skips; an additional actual stdin-worker authentication test passed afterward in 1.444 s. After extracting the layout, 38 deletion-dialog and main-GUI tests passed in 1.477 s and the dialog was rendered for visual inspection. Live deletion fixtures checked every theory-data table, wrong password preservation, rollback after a failure before commit, and reuse of the preserved schema. These counts describe separate runs, not a claimed 371-test combined run. No production database was cleared.

5.10 Theory search: layout, gauge expressions and SQL conditions

The third tab is named **search**. Its editable layout lives in `gui/n2_db.ui`; `SearchTabController` in `gui/search_tab.py` owns the actions, and `gui/theory_search.py` is a separate Sage worker. The default window is 1040×960, enlarged to fit the search layout in the validated offscreen environment. Font scaling and smaller screens may still require scrolling.

Seed theories spans the top. Exact a, c and their lower/upper bounds occupy two columns, followed by **Theory id** and **Only theories with non-empty indices**. CSV field checkboxes are below the conditions. A read-only count/status box and a percentage progress bar sit horizontally above **Search** and **Download**.

Gauge expression	Meaning for one Lagrangian realization
`A1`	Contains at least one A_1 gauge factor.
`A1,C2`	Contains both factors, allowing additional factors.
`"A1,C2"`	Exactly that multiset of gauge factors.
`A1,A1`	Requires at least two A_1 factors.
`"A1";C2`	Exact A_1, or any realization containing C_2.

Factor order is ignored; multiplicities are preserved. SQL uses `EXISTS` with factor counts so all factors of an alternative belong to one realization. Different matching realizations never duplicate a theory ID. Semicolon alternatives are OR-ed; all other supplied conditions are AND-ed.

Charges. Integer, fraction and finite-decimal inputs are parsed as exact `Fractions`. For an exact condition, `0.125` and `2/16` both serialize to `numerator=1,denominator=8`. `JSON_CONTAINS` compares the requested a, c pairs against `central_charges_json` in MySQL. Range bounds are intentionally approximate: endpoints are rounded half-up to 30 decimal places and bound directly to inclusive comparisons on the indexed `central_charge_a_decimal` and `central_charge_c_decimal` columns. Distinct fractions can have the same decimal value. Blank endpoints are unbounded; exact and range predicates can be combined. Invalid input and reversed bounds fail before connecting.

Other conditions. Theory id means `theories.id`, never the realization ID. The index checkbox requires a non-whitespace JSON string in `superconformal_index_json`; Coulomb index, spectrum and precision metadata may be absent. With every condition empty/unchecked, every theory qualifies, including theories without realization or property rows.

The worker disables schema initialization and streams one parameterized SELECT through `SSDictCursor`, returning only ordered theory IDs. It does not load index/input payloads to filter them in Python. Temporary MySQL checks on 18 September confirmed the exact/approximate distinction and range-index access using `EXPLAIN`; they did not access the user's database.

5.11 Retained search results and selectable CSV fields

Search becomes Cancel while the worker runs. Only a successful completed search publishes the immutable `theory_ids` tuple and its count; failed, cancelled or incomplete searches publish no partial snapshot. Filters, database changes/deletion and other-tab work invalidate it. `database` and `conditions` return copied metadata without passwords. Download is disabled initially and enabled after a successful nonempty search. Preferences and conflicting tab work are blocked during operations. Search errors/status appear in the count box tooltip; redacted messages are written under `logs/log_search_*.log`.

Download opens a save-file dialog for the destination name and location. Cancelling that dialog leaves the result available. At least one CSV field must be checked; database identity is rechecked before opening the dialog. The fixed whitelist below is the current implemented selection, in UI order. The theory ID appears once and `properties_json` is excluded.

Source table	CSV column names
`theories`	`theory_id` (from `id`), `name`.
`lagrangian_realizations`	`lagrangian_realization_id` (from `id`), `gauge_group`, `input_json`.
`theory_properties` Basic properties	`flavor_symmetry`, `flavor_rank`, `flavor_dimension`, `conformal_manifold_dimension`, `central_charges_json`, `central_charge_a_decimal`, `central_charge_c_decimal`.
`theory_properties` Indices and cutoffs	`coulomb_branch_index_json`, `coulomb_branch_spectrum_json`, `superconformal_index_json`, `superconformal_index_order`, `coulomb_branch_index_max_dimension_json`.

There are 17 selectable fields. The subsequently added sector count and factor-ID columns (Section 4.9) are stored in the database but have not been added to this whitelist or GUI.

The user selected **one row per Lagrangian realization**. All realizations of each matched theory are exported, even when only one satisfied the gauge filter. Shared properties repeat on those rows. A theory without a realization still has one row, with blank realization fields; missing joined properties are also blank. Rows are ordered by theory ID, then realization ID. SQL NULL becomes an empty cell. Python's CSV writer preserves JSON and exact decimal text and quotes commas, quotes and newlines. The final file is UTF-8 with a BOM and a header containing the selected column names.

The completed search fixes the set of IDs. Export fetches their current values and does not run the search again. Readers may observe external edits at different times; there is no global database snapshot spanning search and download. A searched theory that has disappeared causes failure instead of silent omission. Field selections can change between repeated downloads.

5.12 CSV multiprocessing, progress and atomic publication

`gui/theory_download.py` runs without Sage/Qt imports. A batch contains 32 theory IDs. For $M > 0$ retained theories and resolved CPU budget P:

$$B = \lceil M/32 \rceil , \qquad W_{\mathrm{CSV}} = \min(P, B).$$

The initial four-process limit was removed at the user's request. CPU cores `-1` uses all logical CPUs (fallback one). There is no fixed process cap; the batch count still limits useful workers. The TFORM thread setting does not reduce this export count. One worker uses a serial path; multiple workers use spawned processes with at most one pending batch per worker.

Each reader owns a lazily opened, streaming MySQL connection and writes a CSV chunk to disk. Only row/theory counts cross process boundaries, avoiding large index strings in the parent process. Queries use the fixed column whitelist and bound IDs, with no schema initialization or database writes. Chunks merge in submission order, so serial and parallel output have the same ordering.

If m complete theories have been merged, progress is based on

$$p = 100\, m/M.$$

The denominator counts theories, not realization rows or bytes. Progress can advance in batches and pause behind a slow earlier batch. The last 100% is reserved for successful publication and confirmed worker exit.

A staging directory is created beside the chosen destination. The complete file is flushed and synced, then published with `os.replace` only after every batch succeeds. Failure or cancellation removes staging and preserves any existing destination. Download becomes Cancel; close waits for cooperative cancellation and reader shutdown. Active reads may delay cancellation (reader network timeouts are 30 s). Export locks field selections and conflicting actions while running. Search IDs remain available for another download. Redacted logs use `logs/log_download_*.log`.

Recorded validation. The 18 September suite ran 410 tests (372 passed, 38 skipped). Export fixtures exercised actual spawned readers, CSV fidelity, ordering, progress, cancellation and atomic replacement; Qt fixture processes checked the dialog and controller lifecycle. A separate disposable MySQL server with a SELECT-only account exported 257 theories into 258 rows across all 17 fields. The 7,457,190-byte output matched byte-for-byte between serial and four-reader runs, observed at 0.214 s and 0.164 s. These are single synthetic measurements, not a performance guarantee for the user's data. Missing-theory failure and connection cleanup were also checked. No production data was changed.

6 Package `test` and reproducible use

Selected test modules	Main coverage
`test_check_n2_anomalies.py`	Root/representation data, all families, spectator factors, full/half validation, beta and Witten checks.
`test_n2_theory_index.py`	Laurent output, named/explicit labels, product singlets, low-order reference indices, LiE parsing and cache concurrency.
`test_n2_theory_coulomb_branches.py`	Weyl degrees, repeated/fractional dimensions, PE/PL round trips, relations, full-index projection and precision.
`test_n2_theory_properties.py`	Flavor blocks, central charges, property separation, cutoff metadata, direct Coulomb wiring and exact JSON.
`test_n2_theory_db.py`	Schema/version handling, shared property comparison, full/half deduplication, preservation of existing indices, SQL parameters and transactions; actual index equality through t^6.
`test_n2_theory_db_indices.py`	Deferred worker, independent precision, legacy metadata, concurrent updates, partial retries, rollback, JSON-null selection and real index storage.
`test_form_expansion_cache.py`	Exact term serialization, raw-program keys, thread/process safety, failure behavior and index integration.
`test_cached_decomposition_planning.py`	Reuse of cached composite blocks, base identities and consistent serial/parallel dependency planning.
`test_build_decomposition_cache.py`	Complete enumeration, minimal Adams calls, parallel batches, resume and CLI contracts.

Session coverage includes `test_form_threads.py`, `test_gui_theory_search.py`, `test_gui_search_tab.py`, `test_gui_theory_download.py`, `test_n2_theory_db_sectors.py` and the index search/calculator tests, alongside the earlier disconnected, degree-bound, logging and deletion cases. All paths are relative to `test/`. Migration/backfill tests have been removed; fresh initialization, version rejection and normal sector/index storage remain covered.

6.1 Validation snapshots

The 19 September production-baseline run executed **414 tests in 32.958 s: 374 passed, 40 skipped**, with live MySQL fixtures disabled. An isolated temporary MySQL server then verified fresh schema 1, all ten tables/current columns and repeated initialization. Its 22 integration tests finished in 1.501 s: **20 passed, 2 skipped**; the two skips required a password-protected account. Sector storage, normal independent index upgrades, unknown-cutoff preservation and schema validation were exercised. The server was shut down and removed; the user's database was not accessed.

Earlier session stages separately recorded 340/38 pass/skip for TFORM, 358/38 for SQL search, and 372/38 for CSV export. The export and charge-filter sections describe their additional isolated MySQL checks. These are implementation-time results. This documentation refresh checks source contracts, builds, PDF text and rendered pages; it does not rerun application suites or the expensive benchmarks.

The 14 September deadlock-fix run passed **268 backend tests in 33.518 s, with no skips**, using Sage/FORM/LiE and password-protected disposable MySQL 8.0.46. The GUI documentation-time rerun passed **32 tests in 0.869 s**. Section 5 records the test-harness repairs and real parallel stress checks. Neither run used the user's production database or personal vault.

The earlier two-stage database implementation passed 238 tests in 21.920 s on 12 September, including migrations, concurrent upgrades, rollback, partial retries and real stored indices. That result is retained as historical evidence.

The historical 10 September run recorded 207 passes and two MySQL skips.

6.2 Commands and dependencies

Run from the repository root with Sage's Python, not an unrelated Python interpreter. FORM and LiE must be discoverable for backend tests.

```
QT_QPA_PLATFORM=offscreen PYTHONDONTWRITEBYTECODE=1 \
  sage -python -B -m unittest discover -s test -p 'test_*.py' -v
sage -python -m index.n2_theory_index --help
sage -python -m index.char_decomposition_cache --help
sage -python -m common.n2_theory_properties --help
sage -python -m common.n2_theory_db --help
sage -python -m common.n2_theory_db_indices --help
```

Sage supplies exact rings and Lie data; FORM and LiE handle expansions and representation decompositions. PyMySQL supplies the database connection. The NetworkX dependency supplies gauge-sector connected components. All tests, including GUI fixtures, are under `test/`; see `test/README.md` for focused commands and the opt-in live database/vault checks. The low-level APIs accept executable paths, timeouts and cache paths. The complete Adams-cache builder lazily imports the OR-Tools-based Frobenius solver; ordinary index/cache lookups do not require OR-Tools.

Running on another machine

The session's macOS and LAN MySQL discussion was a portability assessment, not a tested deployment. A second machine needs Sage/PyQt6 and compatible FORM/TFORM and LiE binaries, with its own executable/cache paths. Export and index worker counts use that machine's configured CPU budget.

To use the original PC's MySQL from another LAN host, the client must use the server's LAN address over TCP, not its local Unix socket. The server listener, firewall and a database-scoped account must allow the intended client. No server, firewall or account configuration was changed in this session.

6.3 Practical boundaries before extending the project

Preserve the exact-number contracts, distinct index/spectrum properties and source precision. Non-Lagrangian counting needs explicit ring/freeness assumptions; Higgs work needs an HL/Hilbert-series justification; flavor refinement must preserve characters through gauge projection. These are mathematical contracts.

7 Native implementation options

7.1 Build and select the full-index backend

As of 19 October 2026, `calculate_index` and `calculate_index_internal` default to `backend="native"`. Native C++ performs expansion and singlet projection, with native LiE on character-cache misses. The exact Sage Laurent-polynomial result, input format, sector splitting and optional stored-sector reuse retain their existing contracts. The implementation changes execution and storage, not the index algorithm.

Build once for the Python interpreter used by Sage

Run from the project root. The explicit build produces `index._index_calculation_cpp`, including expansion, projection, character caching and the LiE wrapper:

```
sage -python -B index/cpp/build_index_calculation.py \
  --install-deps --cxx /usr/bin/g++
```

`--install-deps` installs pinned pybind11 and nlohmann/json headers in `index/cpp/_build/deps`; omit it on subsequent builds. The build needs a C++17 compiler, GMP/GMPXX and SQLite development files, plus Make and Bison for the bundled LiE library. `--gmp-prefix` and `--sqlite-prefix` select nonstandard dependency locations. Imports never compile or download. Missing or outdated native builds report the build command; selecting the native full-index backend does not silently fall back to FORM.

Python API options

Option	Meaning for full-index calculation
`backend="native"`	Default: C++ expansion and projection; shared character cache; no FORM expansion-cache reads or writes.
`backend="python"`	Original FORM/TFORM expansion and Python projection. Its character-cache backend is selected independently (next page).
`worker_count=N`	Positive expansion thread count. Overrides `form_threads`; if omitted, the existing `form_threads=1` default applies to either backend.
`processes=N`	Python-backend character-cache process count. It does not parallelize native LiE projection, which remains serial within each calculation.
`timeout=600`	Expansion and individual LiE-operation timeout in seconds. `None` disables deadlines. Native checks are cooperative, with low-overhead polling; this is not a deadline for the complete database/GUI job.
`char_cache_database_path`	Shared SQLite character/singlet cache file; the default is `char_decomposition_cache.db` at the project root.
`form_cache_database_path`	Used only by the Python/FORM backend; defaults beside the character cache. Native expansion is recomputed.

For an existing valid theory dictionary `data`, the return type and serialization stay the same:

```
from index.n2_theory_index import calculate_index

value = calculate_index(data, 18, backend="native",
    worker_count=8, timeout=600,
    char_cache_database_path="characters.db")
reference = calculate_index(data, 18, backend="python",
    worker_count=8, processes=1,
    char_cache_database_path="characters.db")
```

7.2 LiE, compatibility controls and command-line use

The native backend calls `common/cpp/lie.cpp` through the bundled shared library; no LiE executable is needed for the default `lie_executable="lie"` setting. A custom LiE executable path selects subprocess mode, preserving executable wrappers.

Independent character-cache and LiE selections

Set `N2SCFTDB_CHAR_CACHE_BACKEND` before importing the Python character cache module:

Value	Python character-cache behavior
`auto`	Default: use the C++ extension when present; use Python if it is absent. Broken binaries or missing dependencies retain their import errors.
`cpp`	Require the C++ extension.
`python`	Use the original Python cache and executable LiE interface.

This environment variable does not override `backend="native"` for full indices. Selecting `backend="python"` alone does not force a Python cache.

The shared loader is `index._index_calculation_native.load_native`. The full-index path passes `require_index=True` to require expansion/projection support.

For `CharacterDecompositionCache` and `build_decomposition_cache`, the separate `lie_backend` option accepts `"auto"`, `"native"` or `"subprocess"`. Auto uses native LiE with the C++ cache and the default executable setting; a custom executable selects a subprocess. Explicit native mode requires the extension and ignores the executable setting. These constructor/builder options are not additional `calculate_index` arguments.

Command-line examples

```
# Native full-index calculation (also the default backend).
sage -python -B -m index.n2_theory_index theory.json \
  --order 18 --backend native --workers 8 \
  --char-cache-database characters.db

# Original Python cache, executable LiE and FORM/TFORM.
N2SCFTDB_CHAR_CACHE_BACKEND=python \
  sage -python -B -m index.n2_theory_index theory.json \
  --order 18 --backend python --workers 8 --processes 1
```

`--workers` overrides the legacy `--form-threads` option. For the Python backend, one expansion thread selects `--form-executable`; larger counts select `--tform-executable` with `-wN`. The native backend does not use those executables or `--form-cache-database`.

7.3 GUI behavior and engine logs

The GUI uses native full-index calculation by default. Its existing `tools/form_threads` preference also controls native expansion threads. The CPU budget is divided by this thread count, rounding down, to allocate at least one theory worker and at most one worker per selected job. Native LiE projection is serial inside each theory worker. Coulomb-index PE/PL still uses a separate FORM/TFORM executable; Coulomb spectrum calculation uses neither FORM nor LiE.

Startup messages in the Index tab and `logs/log_index_*.log` identify native expansion, shared-library or executable LiE, and separate FORM/TFORM Coulomb expansion with its path and thread count. They describe selected engines; cache or stored-sector reuse can avoid work. Spectrum-only runs report that neither engine is used.

References and equation provenance

Equation numbers in this document are local. Citations near equations identify the primary source or software reference; algebraic recurrences, cutoff rules and function mappings are derived from the inspected project source. Literature section/equation numbers refer to the linked arXiv editions where specified. Base software documentation was consulted on 8 September 2026; the new orthogonality, Sage duality and SQLite references were checked on 10 September. This bibliography also retains the relevant sources from both earlier summaries.

References

[1] The Sage Developers, *Root systems*, SageMath reference manual. Dynkin labels, root/weight lattices and finite Cartan data. Official root-system documentation.

[2] The Sage Developers, *Weyl character rings*, SageMath reference manual. Character degrees, duals and Frobenius–Schur indicators. Official Weyl-character documentation.

[3] L. Bhardwaj and Y. Tachikawa, *Classification of 4d $\mathcal{N} = 2$ gauge theories*, JHEP 12 (2013) 100, updated arXiv edition. arXiv:1309.5160. §2, Eqs. (2.1)–(2.8): matter, spectator dimensions and beta functions.

[4] R. Feger, T. W. Kephart and R. J. Saskowski, *LieART 2.0 – A Mathematica Application for Lie Algebras and Representation Theory*, Comput. Phys. Commun. 257 (2020) 107490. arXiv:1912.10969. Representation tables are cross-checks; LieART is not the project backend.

[5] A. Bilal, *Lectures on Anomalies*. arXiv:0802.0634. Chiral anomaly traces, conjugation and product-group anomaly structure.

[6] E. Witten, *An $SU(2)$ Anomaly*, Phys. Lett. B 117 (1982) 324–328. doi:10.1016/0370-2693(82)90728-6. Conventional four-dimensional mod-two gauge anomaly.

[7] J. Wang, X.-G. Wen and E. Witten, *A New $SU(2)$ Anomaly*, J. Math. Phys. 60 (2019) 052301. arXiv:1810.00844. §2.3 includes the conventional isospin parity formula and distinguishes the additional non-spin anomaly.

[8] I. García-Etxebarria and M. Montero, *Dai–Freed anomalies in particle physics*, JHEP 08 (2019) 003. arXiv:1808.00009. Global-anomaly scope beyond local anomaly polynomials.

[9] A. Gadde, E. Pomoni and L. Rastelli, *The Veneziano Limit of $\mathcal{N} = 2$ Superconformal QCD: Towards the String Dual of $\mathcal{N} = 2$ $SU(N_c)$ SYM with $N_f = 2N_c$*, arXiv:0912.4918. §5, Eqs. (5.4)–(5.5), (5.21)–(5.22): the right-index convention and letters.

[10] S. Benvenuti, B. Feng, A. Hanany and Y.-H. He, *Counting BPS Operators in Gauge Theories: Quivers, Syzygies and Plethystics*, JHEP 11 (2007) 050. arXiv:hep-th/0608050. §5: PE, inverse plethystics, complete intersections and refined counting.

[11] M. A. A. van Leeuwen, A. M. Cohen and B. Lisser, *LiE: A Package for Lie Group Computations* (1992), and LiE reference manual, character operations (Adams and tensor products). TU Eindhoven publication record; CWI manual record.

[12] J. A. M. Vermaseren, J. Davies, T. Kaneko, J. Kuipers, C. Marinissen, B. Ruijl, M. Tentyukov, T. Ueda and J. Vollinga, *FORM version 5.0.0 Reference manual*, 27 January 2026. User-supplied `form-5.0.0-manual.pdf`; see Chapters 1–3 and the entries for symbol restrictions, substitution, sorting and rational functions.

[13] J. Kuipers, T. Ueda, J. A. M. Vermaseren and J. Vollinga, *FORM version 4.0*, Comput. Phys. Commun. 184 (2013) 1453–1467. arXiv:1203.6543. Software background; FORM 5 syntax is taken from the supplied manual.

[14] The Sage Developers, *Finite Coxeter groups*, `degrees()` documentation. Definition as fundamental invariant degrees, including product groups and the repeated D_4 degree. Official finite-Coxeter-group documentation.

[15] A. D. Shapere and Y. Tachikawa, *Central charges of $\mathcal{N} = 2$ superconformal field theories in four dimensions*, JHEP 09 (2008) 109. arXiv:0804.1957. Eqs. (2.5)–(2.6): free multiplets; §3 and Eq. (5.4): Casimir dimensions and $4(2a - c) = \sum_i (2\Delta_i - 1)$, with stated assumptions.

[16] A. Gadde, L. Rastelli, S. S. Razamat and W. Yan, *Gauge Theories and Macdonald Polynomials*, Commun. Math. Phys. 319 (2013) 147–193. arXiv:1110.3740. §4, Eqs. (4.5)–(4.11) and (4.15)–(4.17), Appendix B: HL and Coulomb limits.

[17] Y. Tachikawa, $\mathcal{N} = 2$ *supersymmetric dynamics for pedestrians*, Lecture Notes in Physics 890 (2014). arXiv:1312.2684. Lagrangian, hypermultiplet, flavor and moduli-space background.

[18] C. Córdova, T. T. Dumitrescu and K. Intriligator, *Deformations of Superconformal Theories*, JHEP 11 (2016) 135. arXiv:1602.01217. §3.2.2/Table 22: four-dimensional $\mathcal{N} = 2$ preserving deformations.

[19] M. J. Kang, C. Lawrie, K.-H. Lee, M. Sacchi and J. Song, *Higgs, Coulomb, and Hall–Littlewood*, Phys. Rev. D 106 (2022) 106021. arXiv:2207.05764. Examples and caveats showing that the HL index and Higgs Hilbert series need not coincide, including twisted class-S constructions.

[20] The Stacks Project Authors, *Koszul regular sequences*, Tag 062D. Regular sequences, Koszul homology and exactness criteria used in the constraint-counting derivation.

[21] A. Hanany and R. Kalveks, *Highest Weight Generating Functions for Hilbert Series*, JHEP 10 (2014) 152. arXiv:1408.4690. Representation-level organization of refined Hilbert series.

[22] P. Etingof, *Lie Groups and Lie Algebras*, MIT 18.755 lecture notes (2024), Proposition 32.1 (dual highest weights) and Corollary 35.9 (character orthogonality). Official MIT lecture notes.

[23] SQLite, *Write-Ahead Logging*, especially §2.2 Concurrency. Official WAL documentation.

[24] SQLite, *Datatypes In SQLite*, §2 Storage Classes and Datatypes. Official datatype documentation.

[25] Python Software Foundation, *sqlite3: DB-API 2.0 interface for SQLite databases*; connections, transactions and thread checks. Official Python documentation.

[26] The FORM developers, *FORM 5.0.1 Reference Manual*, 22 June 2026. Official manual; Chapter 4 (modules), Chapter 9 (brackets), and entries for `bracket`, `skip`, `symbols` and `id`. The current-loop derivation is from the project source; this reference documents language semantics. Checked 16 September 2026.

[27] NetworkX developers, *connected_components*, NetworkX reference documentation. Official API reference.

[28] SQLite, *Appropriate Uses For SQLite*. Official deployment guidance.

[29] SQLite, *Temporary Files Used By SQLite*, sections on write-ahead logs and shared-memory files. Official file-lifetime documentation.